

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»
КАФЕДРА ІНФОРМАТИКИ ТА ПРОГРАМНОЇ ІНЖЕНЕРІЇ

КУРСОВА РОБОТА

з дисципліни «Аналіз даних в інформаційних системах»

на тему: «Аналіз зв'язку різних факторів та рівня захворювання туберкульозом
для розв'язання задачі категоризації»

Студента 2 курсу ІП-33 групи

Спеціальності: 121

«Інженерія програмного забезпечення»

Піковського Володимира Мирославовича

«ПРИЙНЯВ» з оцінкою

доц. Ліхоузова Т.А. / доц. Олійник Ю.О.

Підпис

Дата

Київ - 2025 рік

Національний технічний університет України “КПІ ім. Ігоря Сікорського”

Кафедра інформатики та програмної інженерії

Дисципліна Аналіз даних в інформаційно-управляючих системах

Спеціальність 121 "Інженерія програмного забезпечення"

Курс 2 Група ІІІ-33

Семестр 4

ЗАВДАННЯ

на курсову роботу студента

Піковського Володимира Мирославовича

1.Тема роботи Аналіз зв'язку різних факторів та рівня захворювання туберкульозом для розв'язання задачі категоризації

2.Строк здачі студентом закінченої роботи 31.05.2025

3. Вхідні дані до роботи методичні вказівки до курсової робота, обрані дані з сайту

<https://extranet.who.int/tme/generateCSV.asp?ds=estimates>

<https://www.johnsnowlabs.com/marketplace/global-smoking-prevalence-1980-to-2012/>

https://www.kaggle.com/datasets/michaelmatta0/global-development-indicators-2000-2020?select=Global_Development_Indicators_2000_2020.csv

<https://simplemaps.com/data/countries>

4.Зміст розрахунково-пояснювальної записки (перелік питань, які підлягають розробці)

1.Постановка задачі

2.Аналіз предметної області

3.Розробка сховища даних

4.Інтелектуальний аналіз даних

5.Перелік графічного матеріалу (з точним зазначенням обов'язкових креслень)

6.Дата видачі завдання	25.03.2025
------------------------	------------

КАЛЕНДАРНИЙ ПЛАН

№ п/п	Назва етапів курсової роботи	Термін виконання етапів роботи	Підписи керівника, студента
1.	Отримання теми курсової роботи	25.03.2025	
2.	Визначення зовнішніх джерел даних	30.03.2025	
3.	Пошук та вивчення літератури з питань курсової роботи	10.04.2025	
4.	Розробка моделі сховища даних	17.04.2025	
4.	Розробка ETL процесів	27.04.2025	
5.	Обґрунтування методів інтелектуального аналізу даних	1.05.2025	
6.	Застосування та порівняння ефективності методів інтелектуального аналізу даних	15.05.2025	
7.	Підготовка пояснювальної записки	21.05.2025	
8.	Здача курсової роботи на перевірку	31.05.2025	
9.	Захист курсової роботи	02.06.2025	

Студент

Піковський Володимир

Мирославович

(підпис)

(прізвище, ім'я, по батькові)

Керівник

доц. Ліхоузова Т.А

(підпис)

(прізвище, ім'я, по батькові)

Керівник

доц. Олійник Ю.О.

(підпис)

(прізвище, ім'я, по батькові)

АНОТАЦІЯ

Пояснювальна записка до курсової роботи: 62 сторінки, 21 рисунок, 1 таблиця, 15 посилань.

Об'єкт дослідження: датасети з інформацією про туберкульоз та інші показники.

Предмет дослідження: моделі та методи категоризації.

Мета роботи: аналіз зв'язку різних факторів та рівня захворювання туберкульозом для розв'язання задачі категоризації.

Задачі: проектування та реалізація сховища даних, Stage зони та ETL процесів, а також реалізація програмного забезпечення для отримання даних зі сховища та їх подальшого аналізу та моделювання.

Дана курсова робота включає в себе: опис проектування, створення та заповнення сховища даних за даною задачею за допомогою фізичної моделі бази даних; опис створення програмного забезпечення для інтелектуального аналізу даних, їх графічного відображення та розв'язання задачі категоризації за допомогою різних моделей; оцінку якості та порівняльний аналіз побудованих моделей; рекомендації по використанню моделей.

Результати роботи можуть бути корисними для людей, що цікавляться впливом різних факторів на захворюваність.

СХОВИЩЕ ДАНИХ, STAGE ЗОНА, МОДЕЛЬ КЛАСИФІКАЦІЇ, ІНТЕЛЕКТУАЛЬНИЙ АНАЛІЗ ДАНИХ, ФІЗИЧНА МОДЕЛЬ БАЗИ ДАНИХ, ELT ПРОЦЕСИ, МОДЕЛЬ, GRADIENT BOOSTING, RANDOM FOREST, EXTRA TREES, DECISION TREE.

ЗМІСТ

ВСТУП.....	5
1.ПОСТАНОВКА ЗАДАЧІ	6
2.АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ	7
3.РОЗРОБКА СХОВИЩА ДАНИХ	8
3.1 Розробка моделі сховища даних та stage зони	8
3.2 Розробка ETL процесів.....	13
3.3 Завантаження даних за допомогою ETL процесів	14
4.ІНТЕЛЕКТУАЛЬНИЙ АНАЛІЗ ДАНИХ	21
4.1 Обґрунтування вибору методів інтелектуального аналізу даних	21
4.2 Вибір категорій для подальшого застосування моделей.....	21
4.3 Вибір параметрів для вгадування категорії.....	22
4.4 Теоретична основа моделей класифікації	24
4.5 Практичне застосування моделей класифікації	25
4.6 Порівняння ефективності методів інтелектуального аналізу	33
ВИСНОВКИ	38
ПЕРЕЛІК ПОСИЛАНЬ	39
ДОДАТОК А ТЕКСТИ ПРОГРАМНОГО КОДУ.....	41

ВСТУП

В сучасному світі є особливо актуальною тема різноманітних захворювань, що передаються повітряно-крапельним шляхом. Це спричинено нещодавною пандемією вірусу COVID-19, який спричинив глобальний локдаун та паніку. Попри його новизну в рамках історії людства, не варто недооцінювати й старі хвороби, що відомі нам з давніх-давен. Одна з таких – туберкульоз, що є широко поширеною на Землі та спричиняє значний відсоток смертності серед населення.

В рамках даної курсової роботи розроблено сховище даних щодо туберкульозу та інших показників, на основі фізичної моделі бази даних, функціонал якої було розроблено за допомогою SQL скриптів, а саму роботу з базою даних представлено через реалізацію програмного забезпечення для імплементації ETL процесів та інтелектуального аналізу обраних даних.

В ролі системи керування сховищем даних для даної роботи буде виступати MySQL, а мова програмування для реалізації застосунку – Python3.

1.ПОСТАНОВКА ЗАДАЧІ

Під час виконання курсової роботи необхідно виконати наступні завдання:

Створення сховища даних типу «зірка». Структура курсової роботи узгоджена з керівником. Сховище даних повинне містити щонайменше 5 таблиць вимірів та таблицю фактів. Створення ETL процесів для завантаження даних до сховища, створення Stage зони.

Реалізувати спроектоване сховище даних з використанням MySQL версії 8.2.0.

Створення застосунку, що отримує вибірку даних зі створеного сховища, проводить їх інтелектуальний аналіз для отримання передбачення за допомогою різних моделей.

Проведення аналізу вибірки на кореляцію між певними показниками, розподіл даних на категорії.

Для розв'язання задачі категоризації використати наступні моделі: Gradient Boosting, Random Forest, Extra Trees, Decision Tree.

Аналіз отриманих результатів, порівняння різних методів розв'язання задачі категоризації на даній вибірці, отримання найоптимальнішого методу.

Використати мову програмування Python 3 для реалізації застосунку.

Курсовий проект здати до дедлайну (день перед захистом) та виконати у єдиному стилі написання коду (coding style).

2.АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ

Сьогодні теми різних захворювань, які поширюються повітряно-крапельним шляхом, дуже актуальні. Це пов'язано з останньою пандемією COVID-19[1], яка демонструє чутливість глобальної системи до хвороб.

Туберкульоз[2] - це не лише медична, але й соціально-економічна проблема. Його поширення тісно пов'язане зі стандартами проживання населення, доступом до медичних послуг, ВВП на душу населення, індексом людського розвитку, рівнем смертності та епідеміологічних ситуацій ВІЛ/СНІДу.

Аналіз та закономірності поширення туберкульозу на основі таких факторів мають велике значення для системи охорони здоров'я. Це не тільки дозволяє прогнозувати можливі вибухи захворюваності, але й дозволяє ефективно розробити цілеспрямовані стратегії профілактики та лікування. Сучасні підходи до боротьби з туберкульозом потребують інтеграції медичних, соціальних та економічних даних, що дозволяють забезпечити відповідні адміністративні рішення на національному та глобальному рівні.

Сьогодні якісний аналіз факторів, які впливають на поширення туберкульозу, важливий фактор боротьби з цим захворюванням та ключовий крок до побудови стабільної системи охорони здоров'я.

У програмному забезпеченні буде реалізовано наступну функціональність, що включає в себе:

- проектування сховища даних;
- проектування Stage зони
- створення ETL процесів для завантаження і оновлення даних;
- створення вибірки даних з сховища;
- інтелектуальний аналіз даних;
- використання декількох моделей категоризації даних;
- поділ даних по категоріях
- застосування обраних моделей для задачі класифікації
- відображення отриманих результатів та їх аналіз.

3. РОЗРОБКА СХОВИЩА ДАНИХ

3.1 Розробка моделі сховища даних та stage зони

Для виконання курсової роботи було обрано 4 джерела відкритих даних, що включають в себе:

- Інформація про захворювання на туберкульоз:
<https://extranet.who.int/tme/generateCSV.asp?ds=estimates>
- Поширення куріння:
<https://www.johnsnowlabs.com/marketplace/global-smoking-prevalence-1980-to-2012/>
- Показники соціально-економічного розвитку:
https://www.kaggle.com/datasets/michaelmatta0/global-development-indicators-2000-2020?select=Global_Development_Indicators_2000_2020.csv
- Дані про мови та релігії країн: <https://simplemaps.com/data/countries>

Для зручності роботи з наборів були видалені колонки, що нас не цікавлять та зведено дані до загального вигляду. Це було зроблено за допомогою коду мовою програмування Python[3]. Також було змінено назви деяких колонок для зручності. В результаті отримано файли:

1. tuberculosis_stats.csv
2. smoking_stats.csv
3. economic_stats.csv
4. social_stats.csv

Нижче наведений табличний опис вхідних файлів, які безпосередньо використовувались у подальшій побудові бізнес-процесів:

tuberculosis_stats.csv	country	Країна
	year	Рік
	world_region	Регіон світу, де знаходиться країна
	infected_num	Кількість випадків туберкульозу
	population	Населення країни

	infected_with_hiv_num	Відсоток віл-інфікованих серед туберкульозників
	case_detection_rate	Відношення нових випадків за рік
	case_fatality_rate	Смертність
smoking_stats.csv	country	Країна
	smoking_prevalence	Відсоток паліїв
	year	Рік
economic_stats.csv	country	Країна
	year	Рік
	gdp_per_capita	ВВП на душу населення
	health_development_ratio	Рівень розвитку медицини
	human_development_index	IЛР
social_stats.csv	country	Країна
	language	Основна мова
	religion	Найпоширеніша релігія

Таблиця 3.1 – Опис вхідних файлів

На основі аналізу предметної області та отриманих файлів було розроблено наступну модель сховища даних за типом „зірка” (рисунок 3.1).

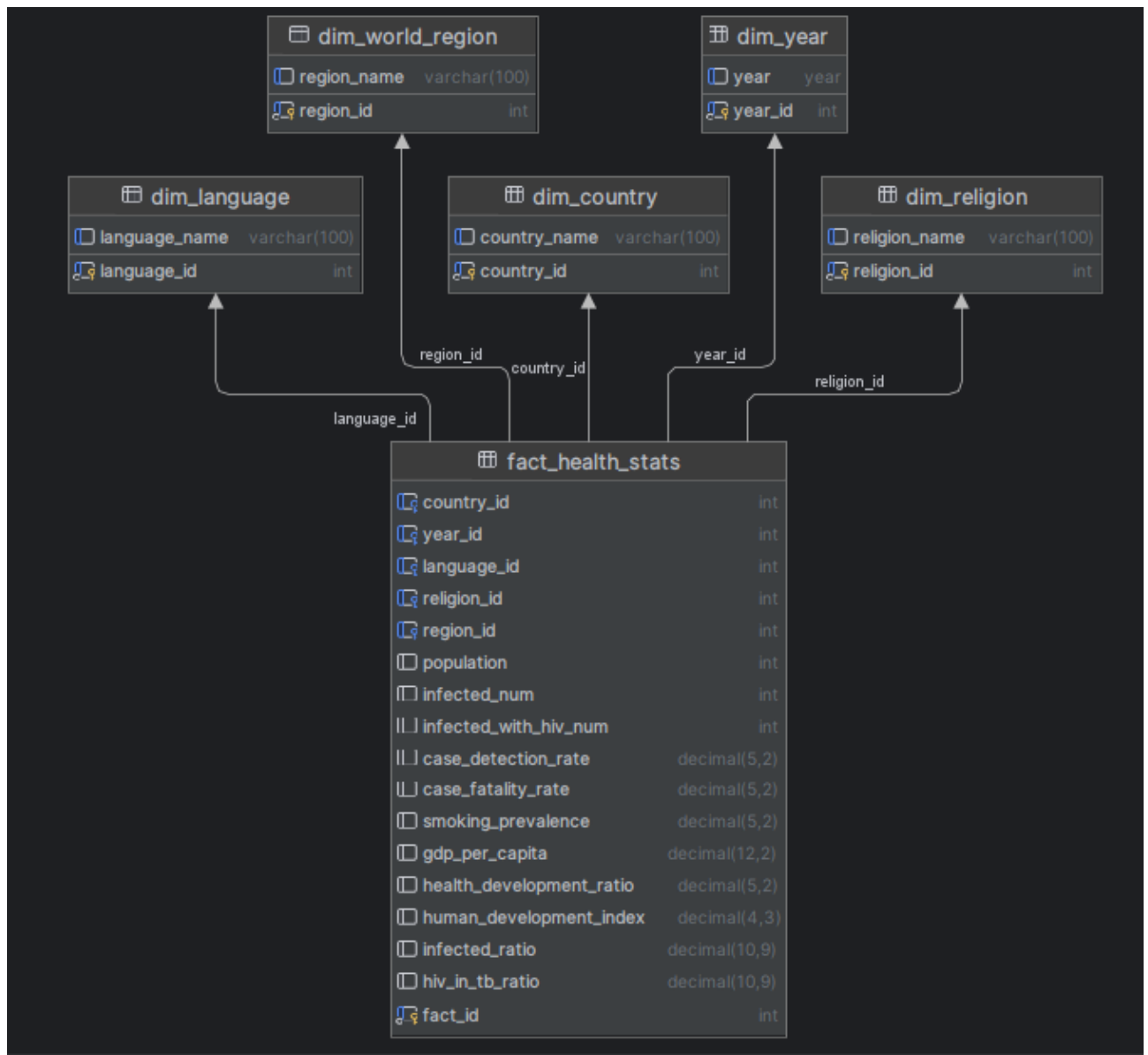


Рисунок 3.1 – Проектування моделі сховища за типом зірка

У моделі сховища за типом зірка спроектовано таблицю фактів та п'ять таблиць вимірів. На даній моделі видно, що центральною таблицею вимірів є таблиця `fact_health_stats`, яка містить зібрану інформацію про населення країн, параметри захворюваності на туберкульоз, статистику куріння, соціально-економічні показники тощо.

Ця таблиця включає дані про захворюваність туберкульозом, рівень смертності від нього тощо та рівень куріння для всіх записів, а також деякі інші дані як загальні показники на країну, такі як ВВП на душу населення. Це дозволяє нам аналізувати ці дані та детальніше зрозуміти зв'язки між ними.

Таблиці dim_year, dim_country, dim_religion, dim_world_region, dim_language містять виміри: роки, країни, релігію, регіон, мову:

- dim_country(country_id, country_name)
- dim_year(year_id, year)
- dim_religion(religion_id, religion_name)
- dim_world_region(region_id, region_name)
- dim_language(language_id, language_name)

На основі спроектованої моделі було створено сховище даних, що включає п'ять таблиць вимірів та таблицю фактів реалізованих за допомогою нижче наведених скриптів використовуючи MySQL версії 8.2.0.

Скрипти створення таблиць в сховищі даних:

```
CREATE TABLE dim_country (  
    country_id INT AUTO_INCREMENT PRIMARY KEY,  
    country_name VARCHAR(100) UNIQUE  
);
```

```
CREATE TABLE dim_year (  
    year_id INT AUTO_INCREMENT PRIMARY KEY,  
    year YEAR UNIQUE  
);
```

```
CREATE TABLE dim_language (  
    language_id INT AUTO_INCREMENT PRIMARY KEY,  
    language_name VARCHAR(100) UNIQUE  
);
```

```
CREATE TABLE dim_religion (  
    religion_id INT AUTO_INCREMENT PRIMARY KEY,  
    religion_name VARCHAR(100) UNIQUE  
);
```

```
CREATE TABLE dim_world_region (  
    region_id INT AUTO_INCREMENT PRIMARY KEY,  
    region_name VARCHAR(100) UNIQUE  
);
```

```
CREATE TABLE fact_health_stats (  
    fact_id INT AUTO_INCREMENT PRIMARY KEY,  
  
    -- Foreign keys  
    country_id INT,  
    year_id INT,  
    language_id INT,  
    religion_id INT,  
    region_id INT,  
  
    -- Indicators  
    population INT,  
    infected_num INT,  
    infected_with_hiv_num INT,  
    case_detection_rate DECIMAL(5,2),  
    case_fatality_rate DECIMAL(5,2),  
    smoking_prevalence DECIMAL(5,2),  
    gdp_per_capita DECIMAL(12,2),  
    health_development_ratio DECIMAL(5,2),  
    human_development_index DECIMAL(4,3),  
  
    -- Foreign key constraints  
    FOREIGN KEY (country_id) REFERENCES dim_country(country_id),  
    FOREIGN KEY (year_id) REFERENCES dim_year(year_id),
```

```

FOREIGN KEY (language_id) REFERENCES dim_language(language_id),
FOREIGN KEY (religion_id) REFERENCES dim_religion(religion_id),
FOREIGN KEY (region_id) REFERENCES dim_world_region(region_id)
);

```

Також перед безпосереднім завантаженням в сховище дані будуть завантажені в stage зону. На рисунку 3.2 наведена її модель.

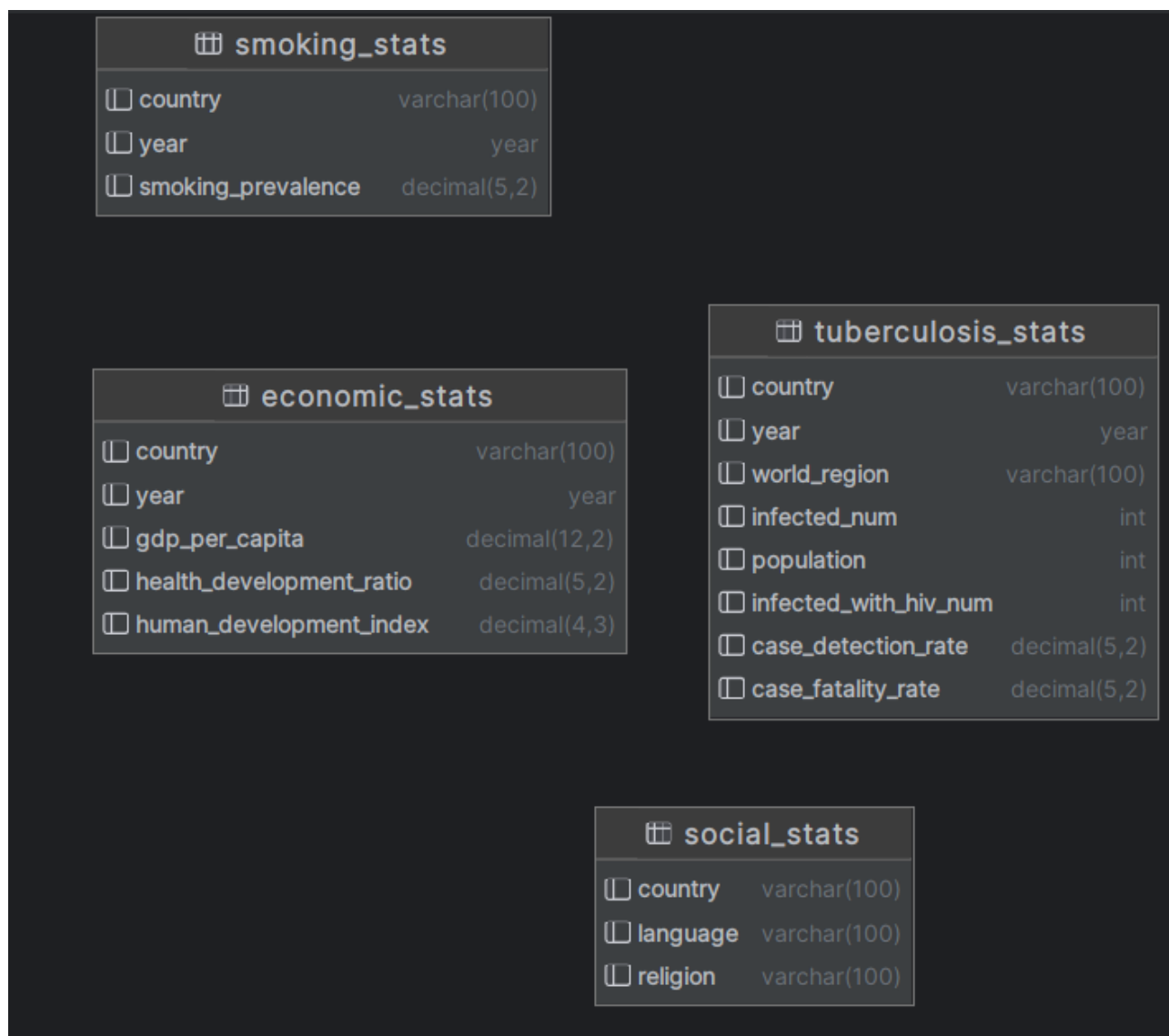


Рисунок 3.2 – Модель Stage зони

Вона по суті копіює будову csv файлів для зручнішого їх зберігання в базі даних.

3.2 Розробка ETL процесів

Для проектування та подальшої роботи з сховищем даних потрібно врахувати комплекс методів, які реалізують процес переносу початкових

даних в аналітичний формат, який буде підтримуватись в сховищі даних та не порушуватиме цілісність системи. Для цього опишемо основні функції ETL процесів, що включають:

Процес попередньої обробки даних.

Цей етап передбачає попередню обробку csv файлів за допомогою коду, перед їх завантаженням в Stage зону.

Процес завантаження даних в Stage.

На цьому етапі головною задачею є перенесення даних з файлів в базу даних.

Перенесення в сховище.

Даний етап ETL процесу потрібний для перенесення даних безпосередньо в саме сховище даних.

Перевірка даних в сховище.

Перевіряємо дані, завантажені в сховище – у випадку помилок виправляємо їх. В результаті дані готові до аналізу.

3.3 Завантаження даних за допомогою ETL процесів

Для заповнення сховища даних було реалізовано наступне:

- 1) Було зведено дані до одного типу, а саме:
 - 1) Зведено назви колонок всіх файлів до одного формату(нижнього регістру)
 - 2) Зведено роки для файлів, де є цей вимір(обрано проміжок 2000-2012)
 - 3) Узгоджено набір країн, для відповідності даних
- 2) Було очищено дані, а саме:
 - 1) Видалено непотрібні для аналізу колонки з файлів
 - 2) Видалено дані щодо статі з даних по курінню(адже більше в жодному з датасетів немає виміру статі)
- 3) Завантажено дані в Stage зону за допомогою функціоналу DataGrip.

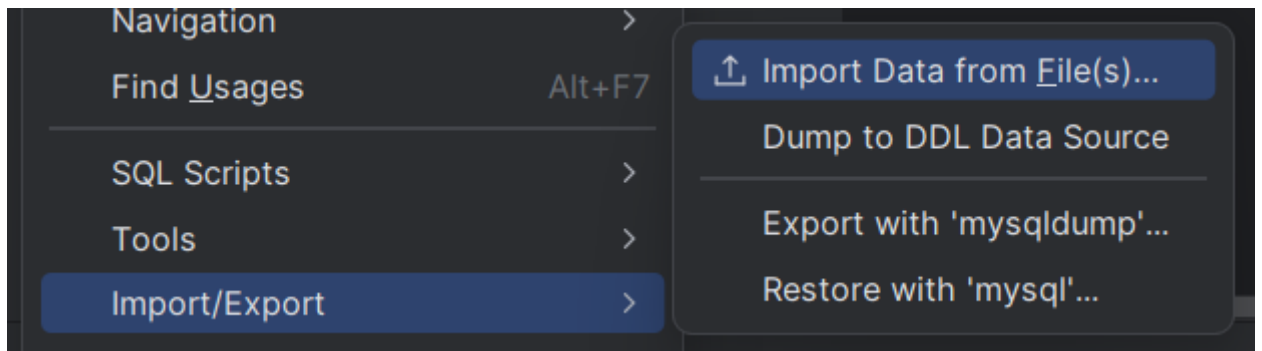


Рисунок 3.3 – Функція імпорту даних

	country	Year	world_region	infected_number	total_population	case_detection_rate	case_fatal
1	Afghanistan	2000	EMR	38000	20130323	19.0	0.37
2	Afghanistan	2001	EMR	38000	20284311	26.0	0.35
3	Afghanistan	2002	EMR	40000	21378110	34.0	0.31
4	Afghanistan	2003	EMR	43000	22733047	32.0	0.32
5	Afghanistan	2004	EMR	44000	23560660	41.0	0.28
6	Afghanistan	2005	EMR	46000	24404569	47.0	0.26
7	Afghanistan	2006	EMR	48000	25424099	53.0	0.24
8	Afghanistan	2007	EMR	49000	25909852	59.0	0.21
9	Afghanistan	2008	EMR	50000	26482615	56.0	0.22
10	Afghanistan	2009	EMR	52000	27466100	50.0	0.25
11	Afghanistan	2010	EMR	54000	28284084	52.0	0.24
12	Afghanistan	2011	EMR	56000	29347711	50.0	0.24
13	Afghanistan	2012	EMR	58000	30560030	50.0	0.25
14	Albania	2000	EUR	700	3166141	86.0	0.03
15	Albania	2001	EUR	640	3147944	86.0	0.03
16	Albania	2002	EUR	680	3134100	87.0	0.03
17	Albania	2003	EUR	640	3118092	87.0	0.03
18	Albania	2004	EUR	630	3098661	87.0	0.04
19	Albania	2005	EUR	580	3076163	87.0	0.02

Рисунок 3.4 – Вигляд таблиці tuberculosis_stats після імпорту в Stage зону

4) Було завантажено дані з Stage до сховища за допомогою SQL скриптів:

```
INSERT IGNORE INTO main_storage.dim_country (country_name)
SELECT DISTINCT country FROM stage.tuberculosis_stats;
```

```
INSERT IGNORE INTO main_storage.dim_year (year)
SELECT DISTINCT year FROM stage.tuberculosis_stats;
```

```
INSERT IGNORE INTO main_storage.dim_language (language_name)
SELECT DISTINCT language FROM stage.social_stats;
```

```
INSERT IGNORE INTO main_storage.dim_religion (religion_name)
SELECT DISTINCT religion FROM stage.social_stats;
```

```
INSERT IGNORE INTO main_storage.dim_world_region (region_name)
SELECT DISTINCT world_region FROM stage.tuberculosis_stats;
```

```
INSERT INTO main_storage.fact_health_stats (
    country_id, year_id, language_id, religion_id, region_id,
    population, infected_num, infected_with_hiv_num,
    case_detection_rate, case_fatality_rate,
    smoking_prevalence, gdp_per_capita, health_development_ratio,
    human_development_index, infected_ratio, hiv_in_tb_ratio
)
```

```
SELECT
    dc.country_id,
    dy.year_id,
    dl.language_id,
    dr.religion_id,
    dwr.region_id,
    tb.population,
    tb.infected_num,
    tb.infected_with_hiv_num,
    tb.case_detection_rate,
    tb.case_fatality_rate,
    sm.smoking_prevalence,
    ec.gdp_per_capita,
    ec.health_development_ratio,
    ec.human_development_index,
    tb.infected_ratio,
    tb.hiv_in_tb_ratio
FROM stage.tuberculosis_stats tb
JOIN stage.smoking_stats sm ON tb.country = sm.country AND tb.year =
sm.year
```

```
JOIN stage.economic_stats ec ON tb.country = ec.country AND tb.year = ec.year
```

```
JOIN stage.social_stats soc ON tb.country = soc.country
```

```
JOIN main_storage.dim_country dc ON tb.country = dc.country_name
```

```
JOIN main_storage.dim_year dy ON tb.year = dy.year
```

```
JOIN main_storage.dim_language dl ON soc.language = dl.language_name
```

```
JOIN main_storage.dim_religion dr ON soc.religion = dr.religion_name
```

```
JOIN main_storage.dim_world_region dwr ON tb.world_region = dwr.region_name;
```

5) Було проведено роботу з даними та виправлено деякі пропуски. А саме:

1) За допомогою скрипту виправлено всі потенційні від'ємні значення:

```
UPDATE main_storage.fact_health_stats
```

```
SET
```

```
population = ABS(population),
```

```
infected_num = ABS(infected_num),
```

```
infected_with_hiv_num = ABS(infected_with_hiv_num),
```

```
case_detection_rate = ABS(case_detection_rate),
```

```
case_fatality_rate = ABS(case_fatality_rate),
```

```
smoking_prevalence = ABS(smoking_prevalence),
```

```
gdp_per_capita = ABS(gdp_per_capita),
```

```
health_development_ratio = ABS(health_development_ratio),
```

```
human_development_index = ABS(human_development_index),
```

```
infected_ratio = ABS(infected_ratio),
```

```
hiv_in_tb_ratio = ABS(hiv_in_tb_ratio);
```

2) Написано скрипт для перевірки даних на пропуски:

```
SELECT
```

```
SUM(population IS NULL) AS null_population,
```

```
SUM(infected_num IS NULL) AS null_infected,
```

```

SUM(infected_with_hiv_num IS NULL) AS null_hiv,
SUM(case_detection_rate IS NULL) AS null_detection,
SUM(case_fatality_rate IS NULL) AS null_fatality,
SUM(smoking_prevalence IS NULL) AS null_smoking,
SUM(gdp_per_capita IS NULL) AS null_gdp,
SUM(health_development_ratio IS NULL) AS null_health,
SUM(human_development_index IS NULL) AS null_hdi
FROM main_storage.fact_health_stats;

```

3) Написано скрипт для заповнення пропусків(середнім значенням по країні або регіону):

```

CREATE TEMPORARY TABLE country_avgs AS

SELECT

country_id,

region_id,

AVG(infected_with_hiv_num) AS avg_infected_with_hiv_num,

AVG(case_detection_rate) AS avg_case_detection_rate,

AVG(case_fatality_rate) AS avg_case_fatality_rate,

AVG(gdp_per_capita) AS avg_gdp_per_capita,

AVG(health_development_ratio) AS avg_health_development_ratio,

AVG(human_development_index) AS avg_human_development_index

FROM fact_health_stats

GROUP BY country_id, region_id;

```

```

CREATE TEMPORARY TABLE region_avgs AS

SELECT

```

```

region_id,

AVG(case_detection_rate) AS reg_avg_case_detection_rate,

AVG(case_fatality_rate) AS reg_avg_case_fatality_rate,

AVG(gdp_per_capita) AS reg_avg_gdp_per_capita,

AVG(health_development_ratio) AS reg_avg_health_dev_ratio,

AVG(human_development_index) AS reg_avg_human_dev_index

FROM fact_health_stats

GROUP BY region_id;

```

```

UPDATE fact_health_stats f

JOIN country_avgs c ON f.country_id = c.country_id

JOIN region_avgs r ON c.region_id = r.region_id

SET

    f.infected_with_hiv_num = IFNULL(f.infected_with_hiv_num,

        IF(c.avg_infected_with_hiv_num IS NULL, 0,

c.avg_infected_with_hiv_num)),

    f.case_detection_rate = IFNULL(f.case_detection_rate,

        IF(c.avg_case_detection_rate IS NULL, r.reg_avg_case_detection_rate,

c.avg_case_detection_rate)),

    f.case_fatality_rate = IFNULL(f.case_fatality_rate,

        IF(c.avg_case_fatality_rate IS NULL, r.reg_avg_case_fatality_rate,

c.avg_case_fatality_rate)),

```

```

f.gdp_per_capita = IFNULL(f.gdp_per_capita,
    IF(c.avg_gdp_per_capita IS NULL, r.reg_avg_gdp_per_capita,
c.avg_gdp_per_capita)),

```

```

f.health_development_ratio = IFNULL(f.health_development_ratio,
    IF(c.avg_health_development_ratio IS NULL,
r.reg_avg_health_dev_ratio, c.avg_health_development_ratio)),

```

```

f.human_development_index = IFNULL(f.human_development_index,
    IF(c.avg_human_development_index IS NULL,
r.reg_avg_human_dev_index, c.avg_human_development_index));

```

```

DROP TEMPORARY TABLE IF EXISTS country_avgs;

```

```

DROP TEMPORARY TABLE IF EXISTS region_avgs;

```

fact_id	country_id	year_id	language_id	religion_id	region_id	population	infected_num
1	1	1	21	4	1	20130323	38
2	5	1	7	5	3	16194869	48
3	2	1	54	4	2	3166141	
4	4	1	68	5	2	65683	
5	7	1	9	5	4	37213982	14
6	8	1	55	5	2	3125627	1
7	6	1	6	5	4	74911	
8	9	1	6	5	5	19132473	1
9	10	1	14	5	2	8012926	1
10	11	1	37	4	2	8174786	6
11	25	1	32	5	3	6470187	19
12	16	1	34	5	2	10251715	1
13	18	1	16	5	3	7221619	6
14	24	1	16	4	3	11925544	8
15	13	1	8	4	6	134544303	298
16	23	1	42	5	2	8000524	4
17	12	1	13	4	1	669580	
18	20	1	52	4	2	4159771	3
19	15	1	39	5	2	9991318	8

Рисунок 3.9 – Вигляд таблиці fact_health_stats після всіх дій

4.ІНТЕЛЕКТУАЛЬНИЙ АНАЛІЗ ДАНИХ

4.1 Обґрунтування вибору методів інтелектуального аналізу даних

Для обробки інформації та виявлення в ній моделей та тенденцій для прийняття рішень необхідно використовувати інтелектуальний аналіз даних. В даній курсовій роботі для аналізу зв'язку різних факторів та захворювання туберкульозом було обрано методи класифікації даних. Розбивши дані на певні категорії та тренуючи моделі відносити дані до цих категорій, можна отримати моделі, що можуть фактично визначати приблизний рівень захворюваності в залежності від інших показників. В якості моделей для проведення розв'язання задачі категоризації було обрано Gradient Boosting, Random Forest, Extra Trees, Decision Tree.

Вибір був зроблений на основі того, що дані моделі гарно працюють з наборами даних без чіткого зв'язку між зміними. Також кожен з цих методів гарно працює з числовими даними та не потребує дуже детальної калібровки як параметрів самого методу, так і даних з набору. Подальший аналіз даних на кожній з моделей та порівняння їх ефективності буде описано в наступних підрозділах.

Також визначимо набір бібліотек, які будуть використовуватися:

- pandas[4]
- numpy[5]
- matplotlib[6]
- seaborn[7]
- scipy[8]
- time9]
- google.colab10]
- sklearn[11]

4.2 Вибір категорій для подальшого застосування моделей

Потрібно було, щоб категорії відповідали обраний темі та обраним даним, тому було прийнято рішення розбити на категорії одну з раніше створених

колонок датафрейму – `infected_ratio`. Було побудовано графік розподілу кількості країн по рівню захворювання на туберкульоз, результати на рисунку 4.1:

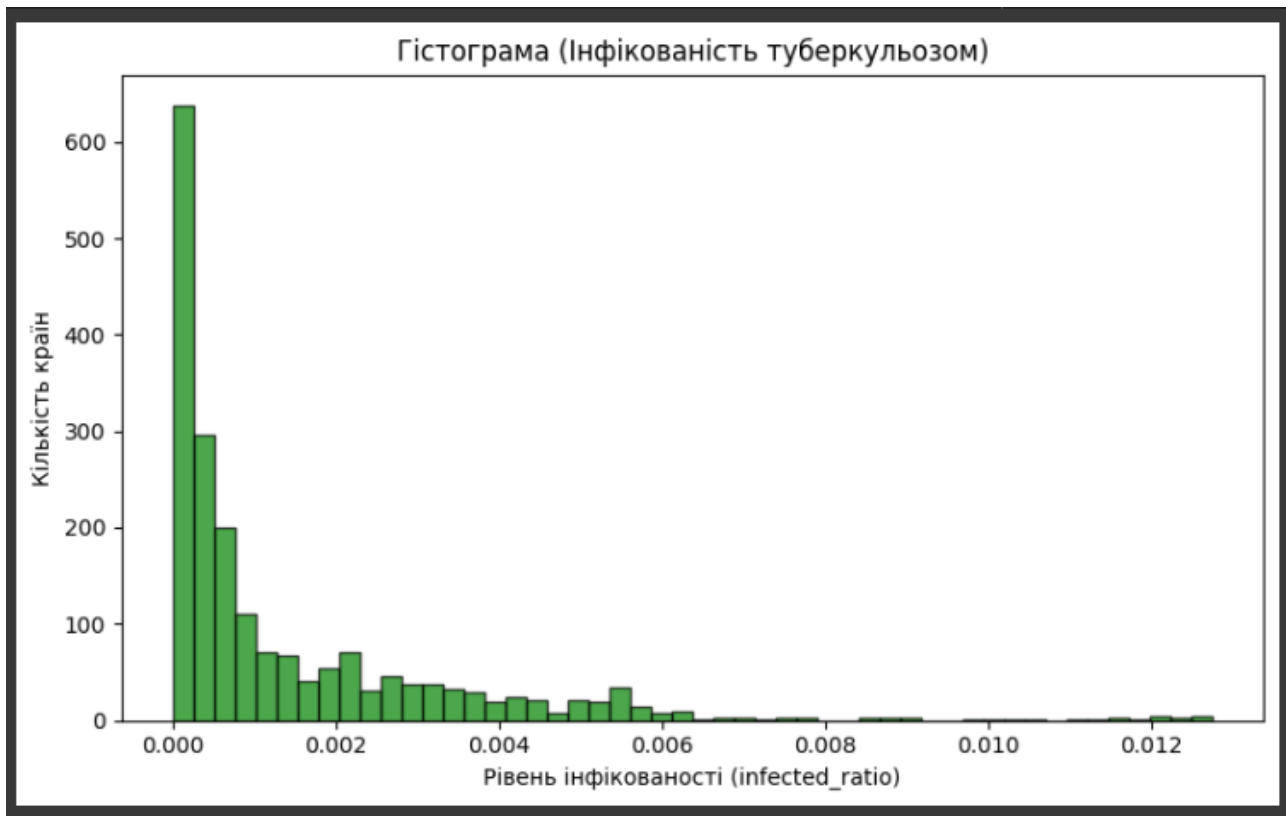


Рисунок 4.1 – Гістограма рівня інфікованості туберкульозом

В результаті аналізу графіку прийнято рішення розподілити категорії наступним чином:

1. $\text{infected_ratio} < 0.001$
2. $\text{infected_ratio} 0.001 \leq 0.003$
3. $\text{infected_ratio} 0.003 \leq 0.005$
4. $\text{infected_ratio} \geq 0.005$

4.3 Вибір параметрів для вгадування категорії

Тепер потрібно провести дослідження числових параметрів датафрейму та визначити, які з них найкраще буде включити до навчальних та тестових даних при застосуванні моделей для класифікації. Результат на рисунку 4.2:

The Pearson Correlation Coefficient between 'infected_ratio' and 'population' is 0.0543 with a P-value of 0.01572
The Pearson Correlation Coefficient between 'infected_ratio' and 'case_detection_rate' is -0.5820 with a P-value of 1.702e-179
The Pearson Correlation Coefficient between 'infected_ratio' and 'case_fatality_rate' is 0.4033 with a P-value of 3.657e-78
The Pearson Correlation Coefficient between 'infected_ratio' and 'smoking_prevalence' is -0.1137 with a P-value of 4.039e-07
The Pearson Correlation Coefficient between 'infected_ratio' and 'gdp_per_capita' is -0.3370 with a P-value of 1.092e-53
The Pearson Correlation Coefficient between 'infected_ratio' and 'health_development_ratio' is -0.4491 with a P-value of 1.157e-98
The Pearson Correlation Coefficient between 'infected_ratio' and 'human_development_index' is -0.5181 with a P-value of 3.711e-136
The Pearson Correlation Coefficient between 'infected_ratio' and 'hiv_in_tb_ratio' is 0.5869 with a P-value of 3.127e-183
The Pearson Correlation Coefficient between 'infected_ratio' and 'language_id' is -0.1710 with a P-value of 1.963e-14
The Pearson Correlation Coefficient between 'infected_ratio' and 'religion_id' is 0.1308 with a P-value of 5.338e-09
The Pearson Correlation Coefficient between 'infected_ratio' and 'region_id' is 0.1911 with a P-value of 1.038e-17

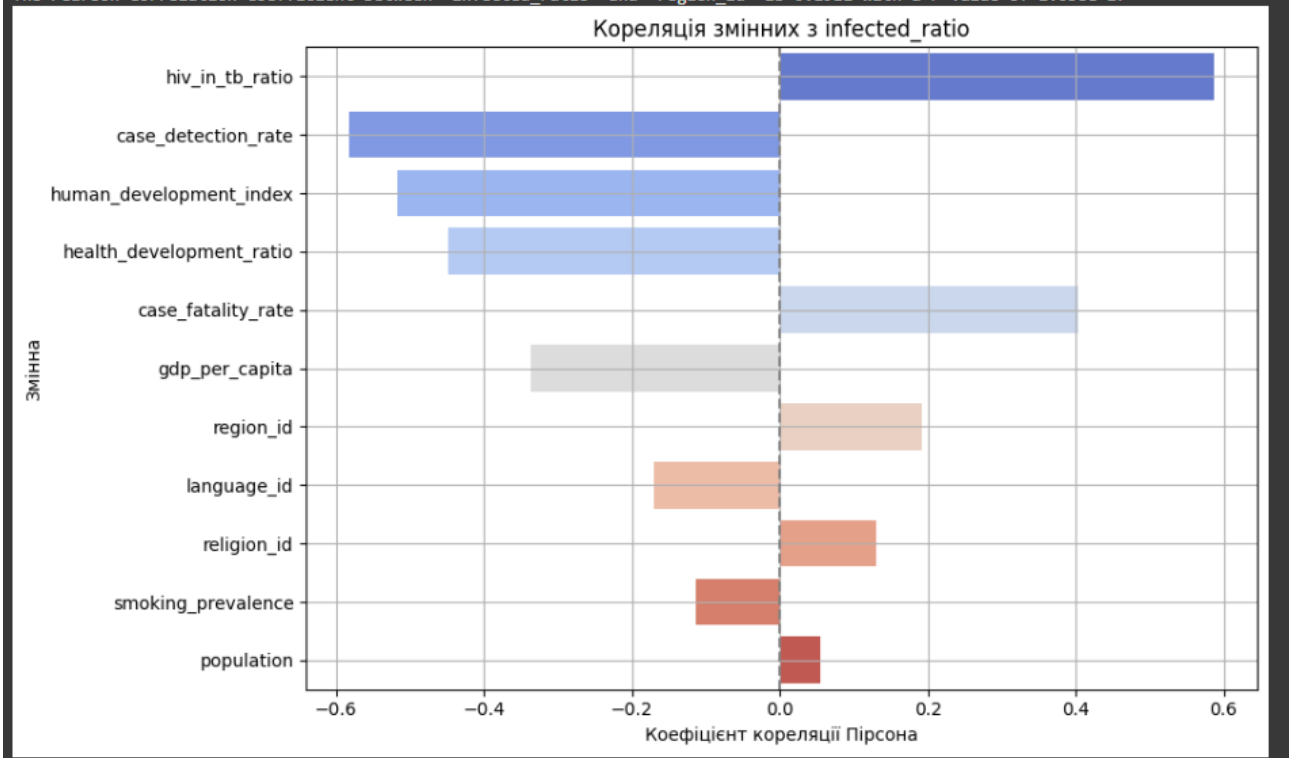


Рисунок 4.2 – Кореляція показників та infected_ratio

Як можна побачити з графіку, найбільше з відсотком заражених корелює відсоток заражених на віл серед хворих на туберкульоз, відсоток знаходження випадків захворювання та індекс людського розвитку. Також непогано корелюють індекс розвитку медицини, фатальність та ВВП на душу населення. Інші ознаки виявили малу кореляцію, отже, не підходять для аналізу. Відкинемо їх та зробимо вибірки на основі параметрів, що гарно корелюють. Формування вибірок на рисунку 4.3:

```

# Готуємо дані до задачі класифікації, будемо намагатися визначити infected_level
# Беремо ті параметри, що найбільше корелюють з infection_rate
all_features=pd.get_dummies(df[['case_detection_rate',
'case_fatality_rate', 'health_development_ratio', 'human_development_index', 'hiv_in_tb_ratio']])
all_features[['infected_level']] = df[['infected_level']]

# Ділимо дані на навчальну та тестову вибірки
df_train, df_test = train_test_split(
    all_features,
    test_size=0.2,
    random_state=1
)

x_train = df_train[['case_detection_rate',
'case_fatality_rate', 'health_development_ratio', 'human_development_index', 'hiv_in_tb_ratio']]
y_train = df_train[['infected_level']]

x_test = df_test[['case_detection_rate',
'case_fatality_rate', 'health_development_ratio', 'human_development_index', 'hiv_in_tb_ratio']]
y_test = df_test[['infected_level']]

```

Рисунок 4.3 – Розподіл даних на навчальну та тестову вибірки

4.4 Теоретична основа моделей класифікації

Розглянемо теорію моделей, обраних для цієї роботи.

Gradient Boosting - це алгоритм, який робить точні прогнози, об'єднуючи кілька дерев рішень у одну модель. Цей спосіб для моделювання використовує сильні сторони базових моделей, виправляючи помилки і поліпшуючи здатності прогнозування. Вловлюючи складні патерни в даних Gradient Boost добре виконує різні завдання моделювання. [12].

Цей алгоритм зацікавив мене своєю складністю та точністю, він дає гарні результати але потребує довгого часу виконання, особливо на великих наборах даних.

Random Forest – це алгоритм, що також працює на основі дерев рішень, для кожного з них випадковим образом обираючи дані для аналізу. Далі поєднує їхні результати шляхом голосування для класифікації або усереднення для регресії[13]

Цей алгоритм є вдалим для нашого набору даних, оскільки він гарно себе показує на великих наборах даних, має високу точність та не дуже схильний до перенавчання. Ще він гарно показує себе на наборах де частина даних відсутня, але це не наш випадок, оскільки пропуски були заповнені.

Extra Trees, подібно до випадкових лісів є підходом, який навчає численні дерева рішень і агрегує результати з групи дерев рішень для отримання прогнозу. Однак між Extra Trees і Random Forest є кілька відмінностей.

Random Forest використовує Bootstrap aggregating для вибору різних варіацій навчальних даних, щоб забезпечити достатню відмінність дерев рішень. Однак Extra Trees використовує весь набір даних для навчання дерев рішень. Таким чином, щоб забезпечити достатню різницю між окремими деревами рішень, він випадково обирає значення, при яких потрібно розділити ознаку і створити дочірні вузли. На противагу цьому, у випадковому лісі ми використовуємо алгоритм жадібного пошуку і вибираємо значення, на якому потрібно розділити ознаку[14].

Відмінність в результатах – більша швидкість, ніж в random forest.

Decision Tree – досить базовий алгоритм. Робота дерева рішень починається з головного питання, відомого як кореневий вузол. Це питання впливає з особливостей набору даних і слугує відправною точкою для прийняття рішень.

Від нього дерево задає серію запитань з відповідями «так» і «ні». Кожне питання покликане розділити дані на підмножини на основі певних атрибутів. Якщо ваша відповідь «Так», ви можете піти одним шляхом, якщо «Ні», ви підете іншим шляхом. В кінці на останній гілці дається категорія(або робить прогноз, якщо дерево використано для нього)[15].

Це простий для розуміння алгоритм, на основі якого будуються деякі з описаних раніше. Він є базовим, з його переваг – гарна робота з ненормованими даними та нелінійними відношеннями. Мінуси – перенавчання, нестабільність.

4.5 Практичне застосування моделей класифікації

Спочатку проведемо підбір параметрів для наших моделей. Від параметрів досить сильно залежить точність моделі, і саме по точності ми будемо визначати яке саме значення параметру є найбільш вдалим для подальшого використання.

Першою є модель Gradient Boosting. Для підбору оберемо параметри `n_estimators`, `learning_rate` і `max_depth`. Обрано саме три для того, щоб сильно не

збільшувати час підбору параметрів, оскільки при досягненні певного рівня точності далі крок вже мінімальний. N-estimators – це кількість дерев, що будуються. Чим вище кількість, тим точніша модель, але одночасно зростає ризик перенавчання та час обрахунку. Learning rate – швидкість, з якою відбувається навчання моделі, робить процес навчання повільнішим, але надійнішим. Отже, однією з цілей підбору параметрів є знаходження балансу між цими двома параметрами. Третій – max_depth – максимальна глибина дерева, потрібно також балансувати між великими(ризик перенавчання) та низькими(ризик недонавчання) значеннями.

Оскільки параметрів три, застосовано метод сітки GridSearchCV. Код та результати виконання на рисунку 4.4:

```
# Підбираємо параметри для GradientBooster
gb = GradientBoostingClassifier(random_state=1)

param_grid = {
    'n_estimators': [100],
    'learning_rate': [0.05, 0.1],
    'max_depth': [3, 5]
}
grid_search = GridSearchCV(estimator=gb, param_grid=param_grid,
                           cv=5, scoring='accuracy', n_jobs=-1, verbose=2)
grid_search.fit(x_train, y_train)

# Виведення результатів
print("Best parameters found:", grid_search.best_params_)
print("Best cross-validation accuracy:", grid_search.best_score_)

Fitting 5 folds for each of 4 candidates, totalling 20 fits
Best parameters found: {'learning_rate': 0.1, 'max_depth': 5, 'n_estimators': 100}
Best cross-validation accuracy: 0.870253164556962
```

Рисунок 4.4 – Результати підбору параметрів для моделі Gradient Boosting

В результаті отримали такий набір найкращих параметрів – n_estimators = 100, learning_rate = 0.1, max_depth = 5.

Проте, як можна побачити на рисунку, спочатку був заданий ще один параметр – random_state. Це спосіб зафіксувати деякі випадкові події, несильно,

але все ж певним чином впливає на точність. За допомогою простенького циклу та графіку виберемо найкраще значення цього параметру на рисунку 4.5:

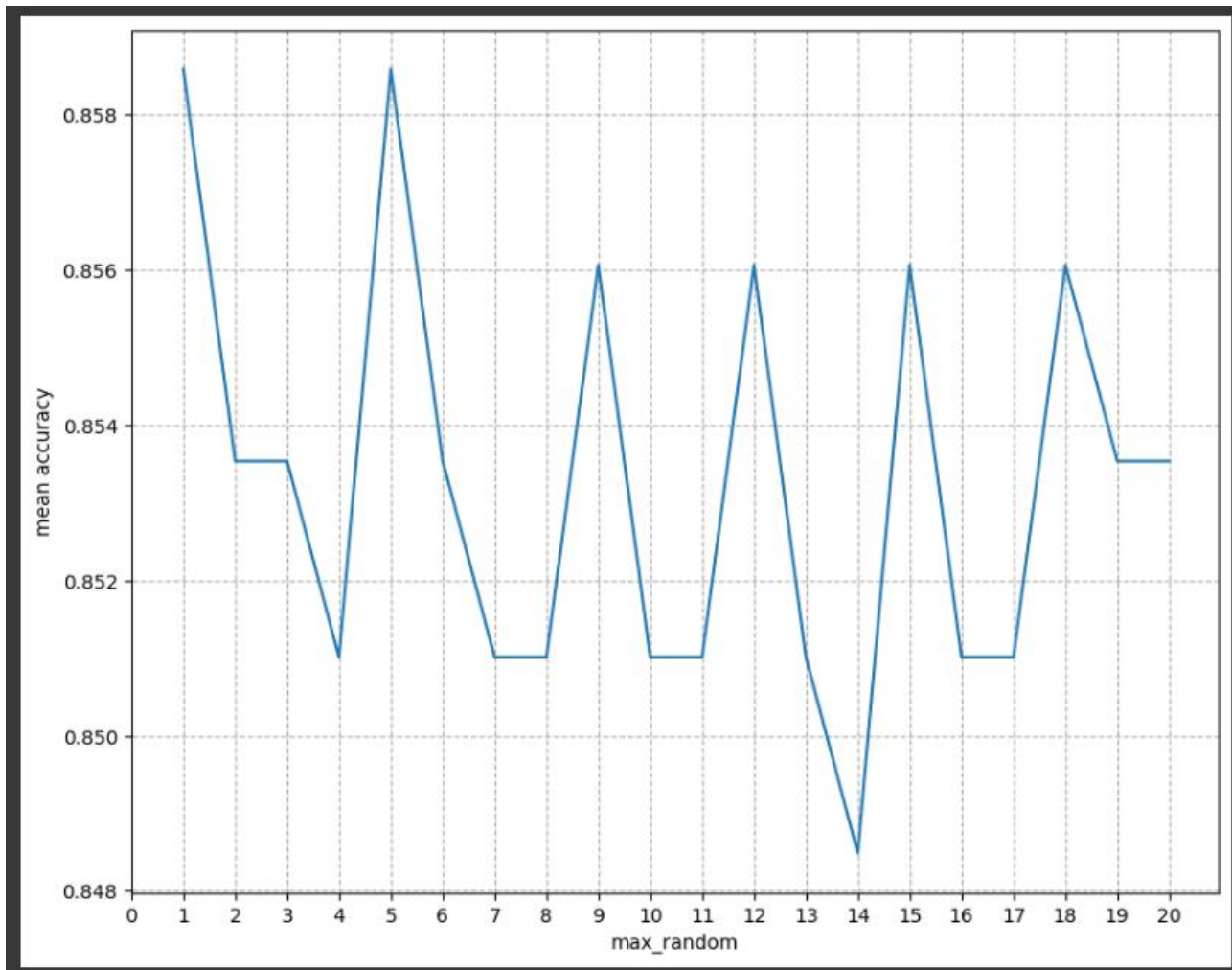


Рисунок 4.5 – Графік зміни точності залежно від random_state

Як бачимо, можна залишити для random_state значення 1.

Для другої моделі – Random Forest – виконано підбір параметрів max_depth та random_state. Перший параметр – це максимальна глибина дерева, перевіряємо до якого моменту має сенс збільшувати глибину, а після якого вже починається лише ускладнення без особливої вигоди в точності. Другий параметр – random_state – це спосіб зафіксувати деякі випадкові події, не сильно, але все ж певним чином впливає на точність.

Спочатку за допомогою звичайного циклу підберемо глибину, допустивши значення `random_state = 1`, та побудуємо графік щоб обрати найкращий. Результати на рисунку 4.6:

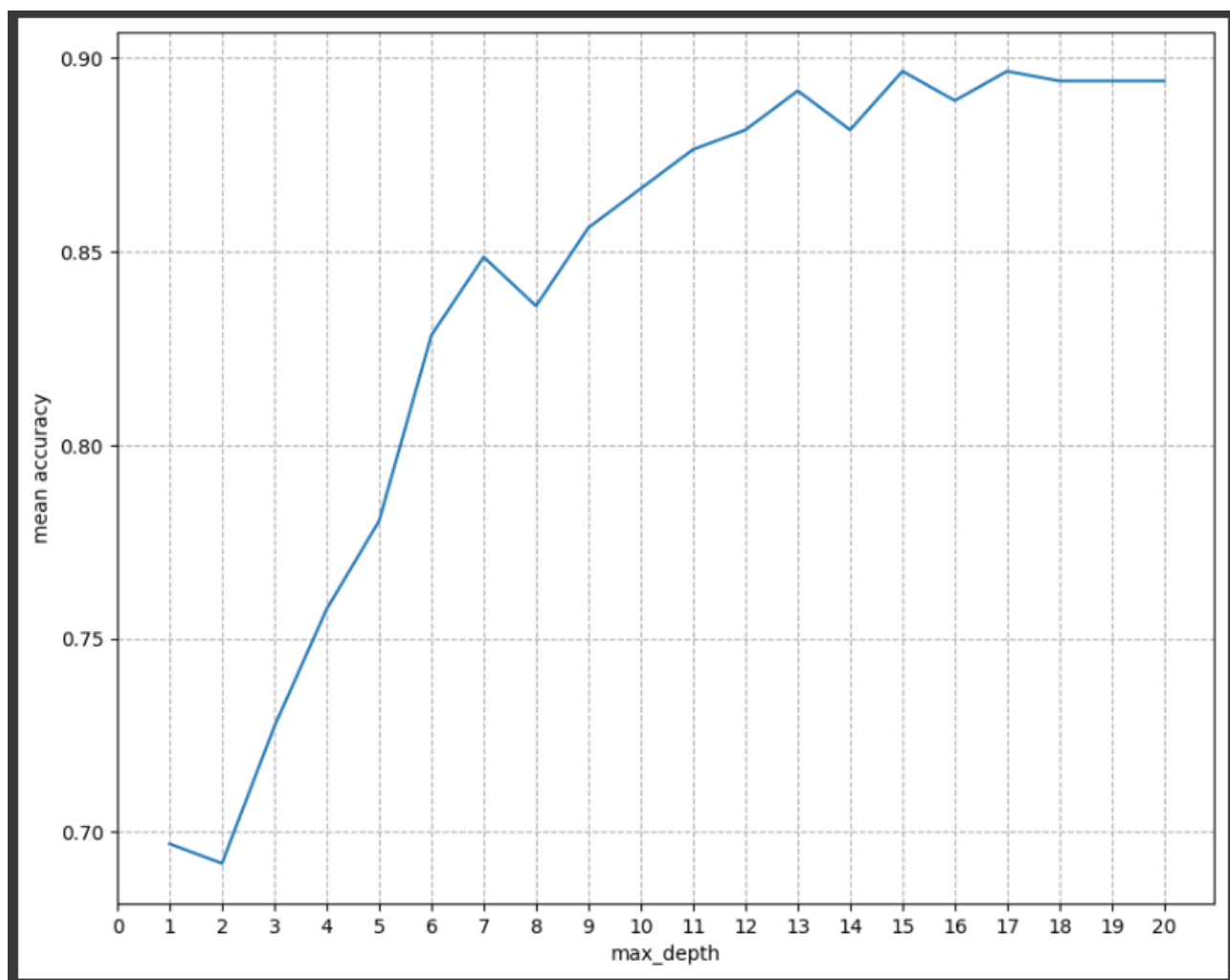


Рисунок 4.6 – Результати підбору `max_depth`

Як бачимо, оптимальне значення – 7.

Тепер аналогічним чином підберемо `random_state`, вже знаючи, що оптимальне значення для першого параметру – 7. Результати на рисунку 4.7:

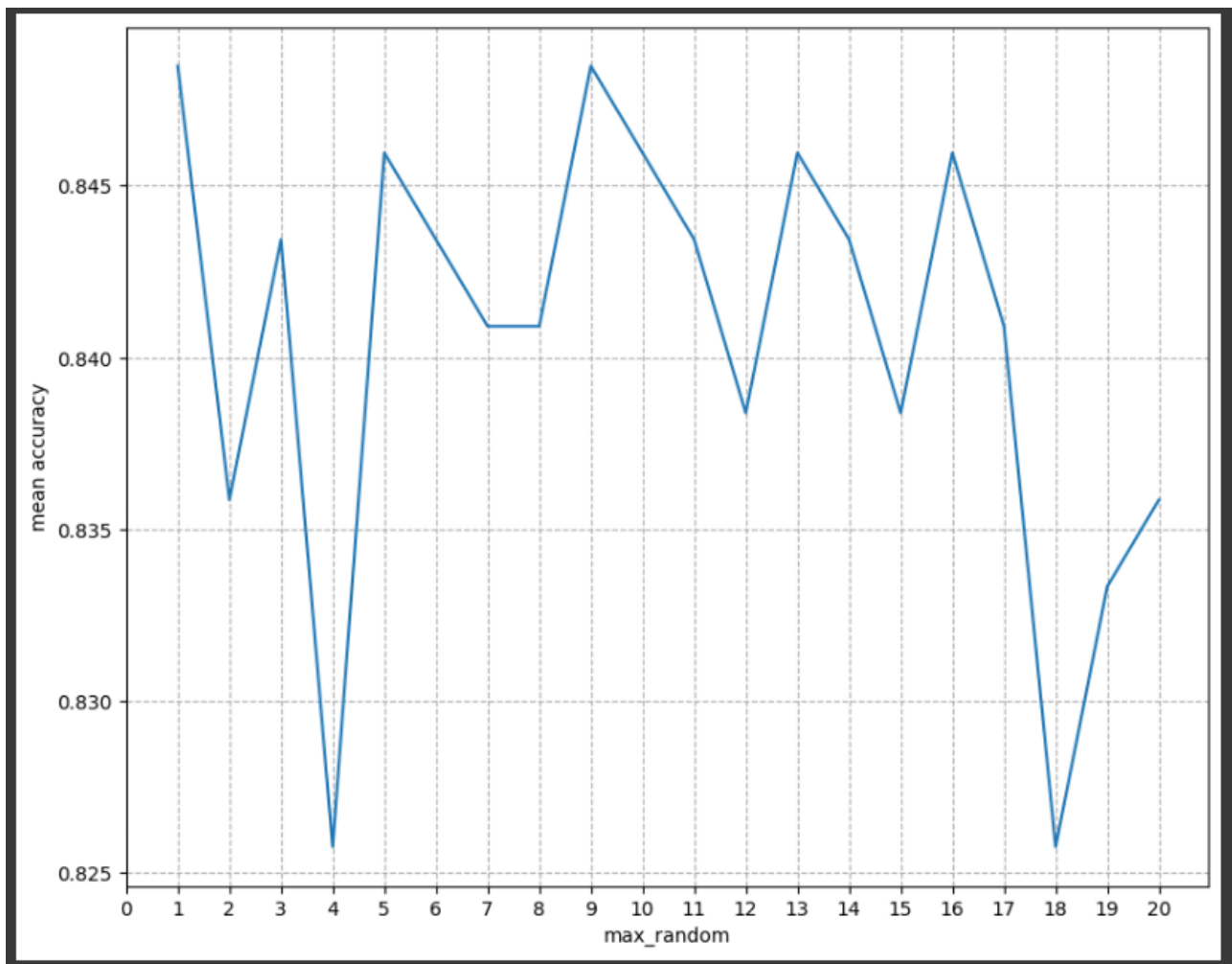


Рисунок 4.7 – Результати підбору random_state

Можна зробити висновок, що оптимальні параметри для цієї моделі це max_depth=7, random_state=1.

Для третьої моделі – Extra trees – виконано підбір параметрів max_depth та random_state. Перший параметр, аналогічно з попередньою моделлю, це максимальна глибина дерева, перевіряємо до якого моменту має сенс збільшувати глибину, а після якого вже починається лише ускладнення без особливої вигоди в точності. Другий параметр – random_state – це спосіб зафіксувати деякі випадкові події, не сильно, але все ж певним чином впливає на точність.

Спочатку за допомогою звичайного циклу підберемо глибину, допустивши значення random_state = 1, та побудуємо графік щоб обрати найкращий. Результати на рисунку 4.8:

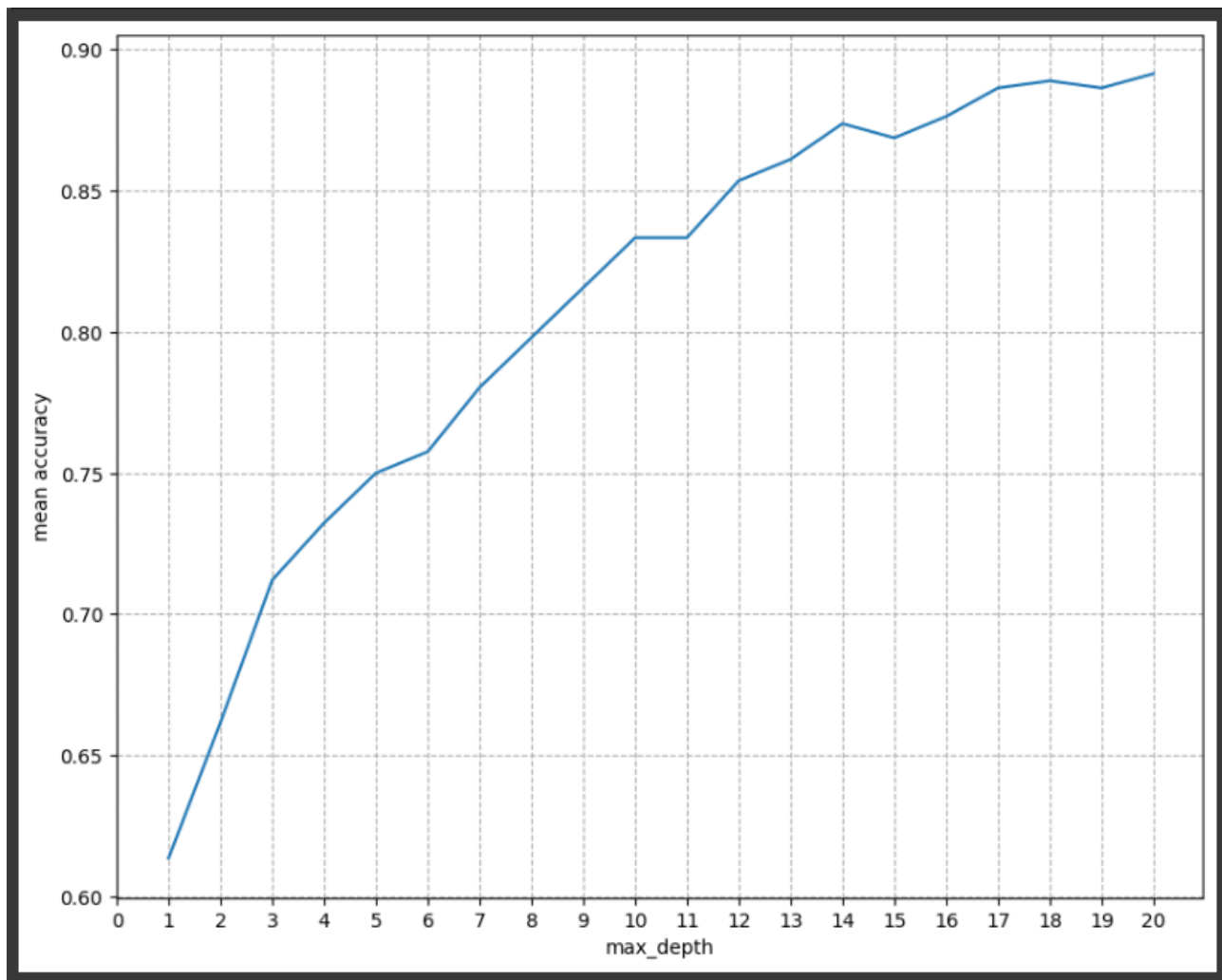


Рисунок 4.8 – Результати підбору max_depth

Як бачимо, оптимальне значення – 10.

Тепер аналогічним чином підберемо random_state, вже знаючи, що оптимальне значення для першого параметру – 10. Результати на рисунку 4.9:

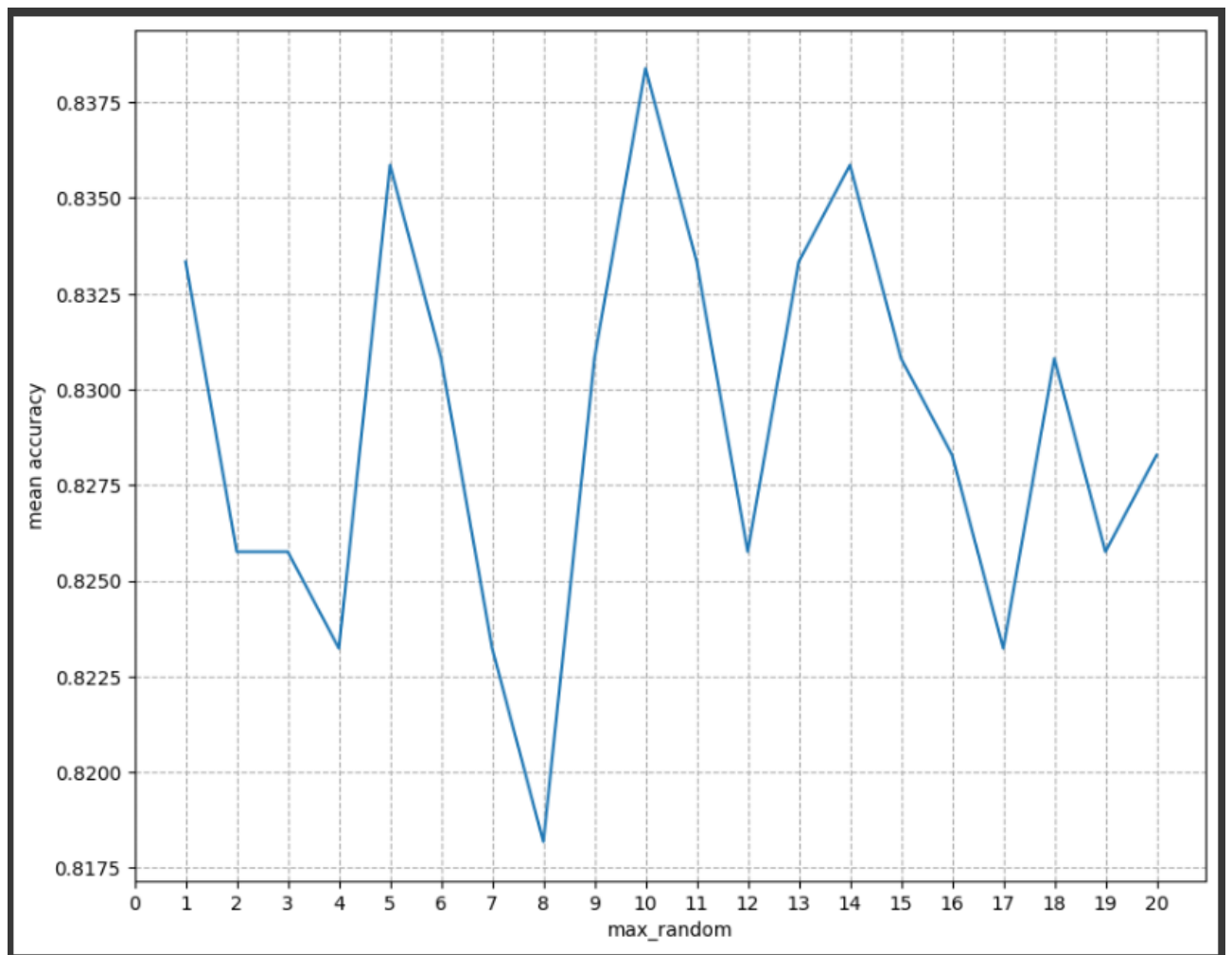


Рисунок 4.9 – Результати підбору random_state

Можна зробити висновок, що оптимальні параметри для цієї модель це max_depth=10, random_state=10.

Остання модель – Decision Tree. Розглянемо ті самі параметри, що й для попередніх двох моделей. Результати першого циклу підбору на рисунку 4.10:

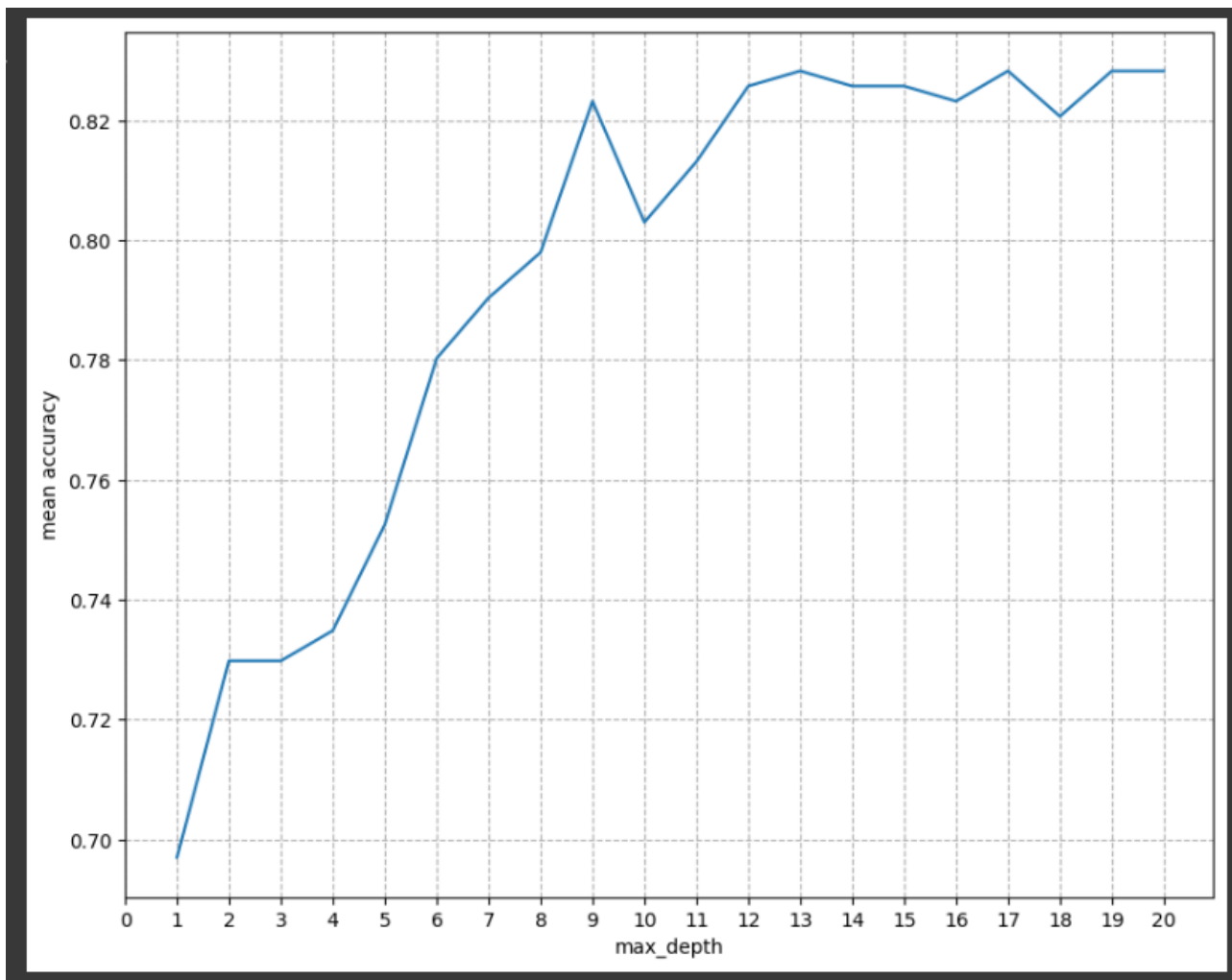


Рисунок 4.10 – Результати підбору max_depth

Тепер враховуючи що найкраще max_depth = 9, підберемо другий параметр, графік на рисунку 4.11:

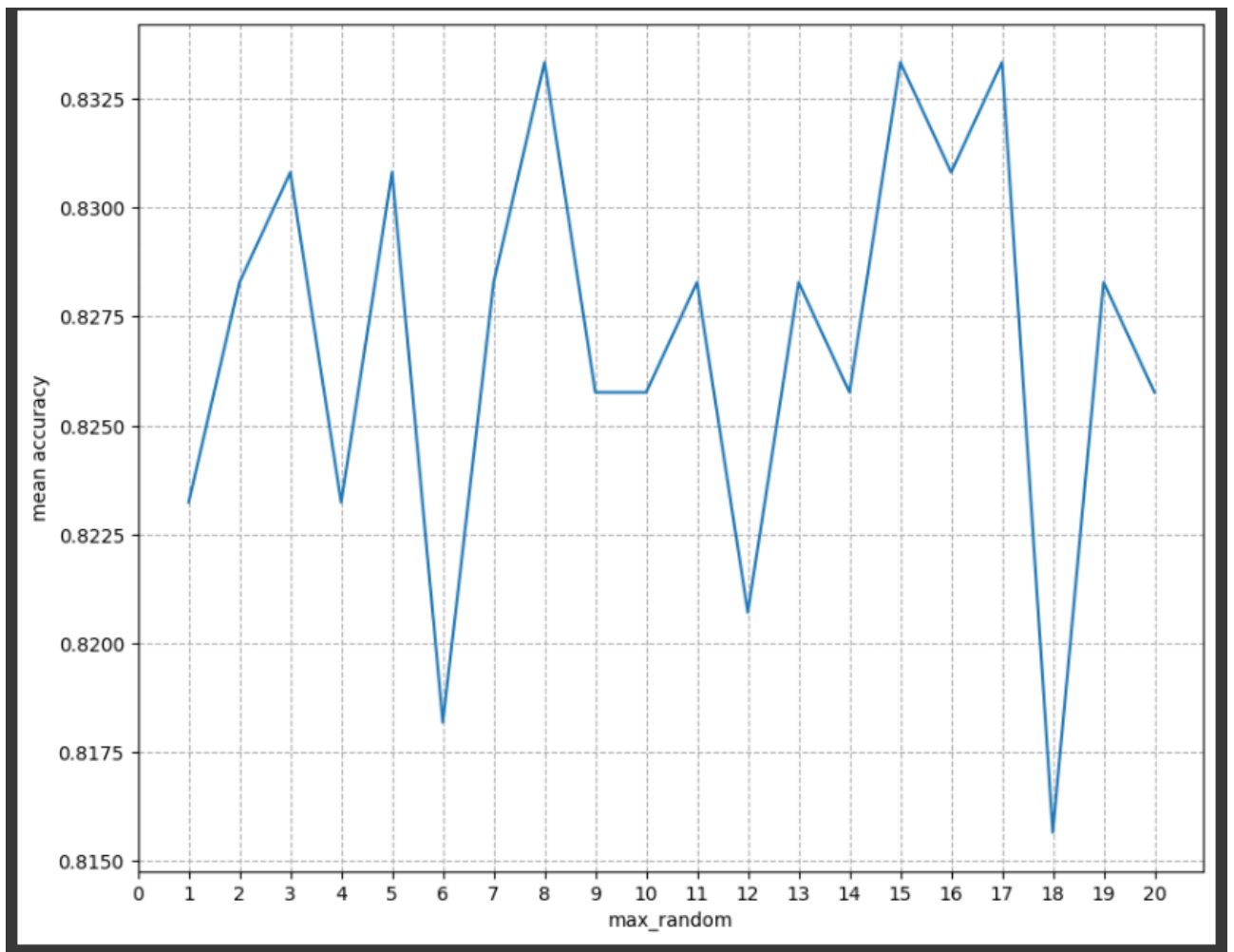


Рисунок 4.11 – Результати підбору random_state

Можна зробити висновок, що оптимальні параметри для цієї моделі це max_depth=9, random_state=8.

Враховуючи вищеписану реалізацію можна зробити висновки, проаналізувавши дані, провести порівняльні тести та вибрати найоптимальніший метод.

4.6 Порівняння ефективності методів інтелектуального аналізу

Для аналітичного визначення точності результатів, отриманих на основі моделей для категоризації, та вибору найоптимальнішого методу знайдемо показники f1_score та точність(score) для кожної з моделей на навчальних та тренувальних даних. Також засічемо час для кожної моделі. Результат на рисунку 4.12:

```
Accuracy on training data
Gradient Boosting : 0.9994 (Time: 0.1048 sec)
Random Forest : 0.9184 (Time: 0.0390 sec)
Extra Trees : 0.9310 (Time: 0.0631 sec)
Decision Tree : 0.9418 (Time: 0.0187 sec)

Accuracy on test data
Gradient Boosting : 0.8535 (Time: 0.0138 sec)
Random Forest : 0.8485 (Time: 0.0232 sec)
Extra Trees : 0.8384 (Time: 0.0470 sec)
Decision Tree : 0.8333 (Time: 0.0054 sec)

F1 Score on training data
Gradient Boosting : 0.9994 (Time: 0.0802 sec)
Random Forest : 0.9192 (Time: 0.0737 sec)
Extra Trees : 0.9304 (Time: 0.0974 sec)
Decision Tree : 0.9415 (Time: 0.0325 sec)

F1 Score on test data
Gradient Boosting : 0.8544 (Time: 0.0542 sec)
Random Forest : 0.8489 (Time: 0.0737 sec)
Extra Trees : 0.8378 (Time: 0.1014 sec)
Decision Tree : 0.8295 (Time: 0.0182 sec)
```

Рисунок 4.12 – Результат тестування моделей

Також побудовано декілька графіків для наглядного відображення результатів, адже результати трохи варіюються між різними виконаннями коду, на рисунках 4.13-4.16:

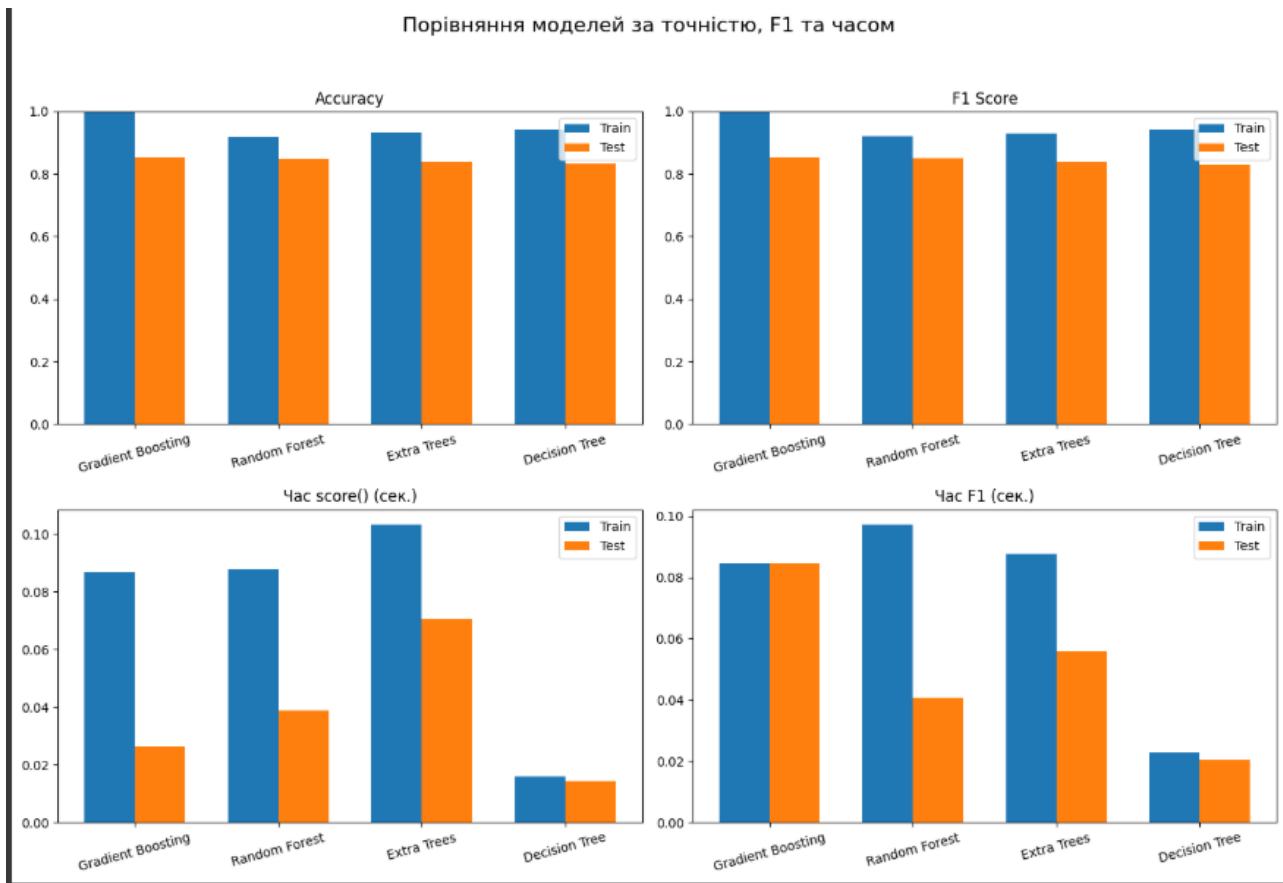


Рисунок 4.13

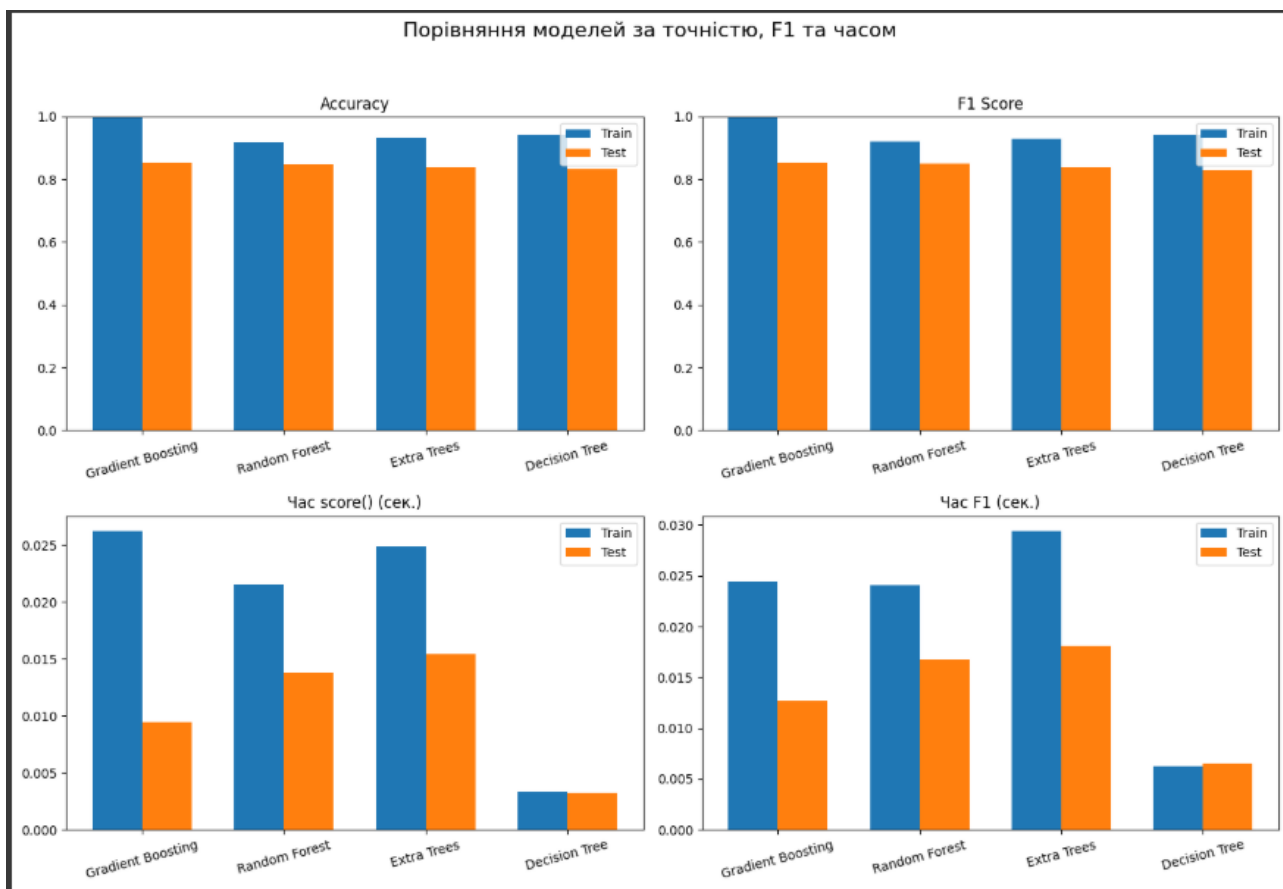


Рисунок 4.14

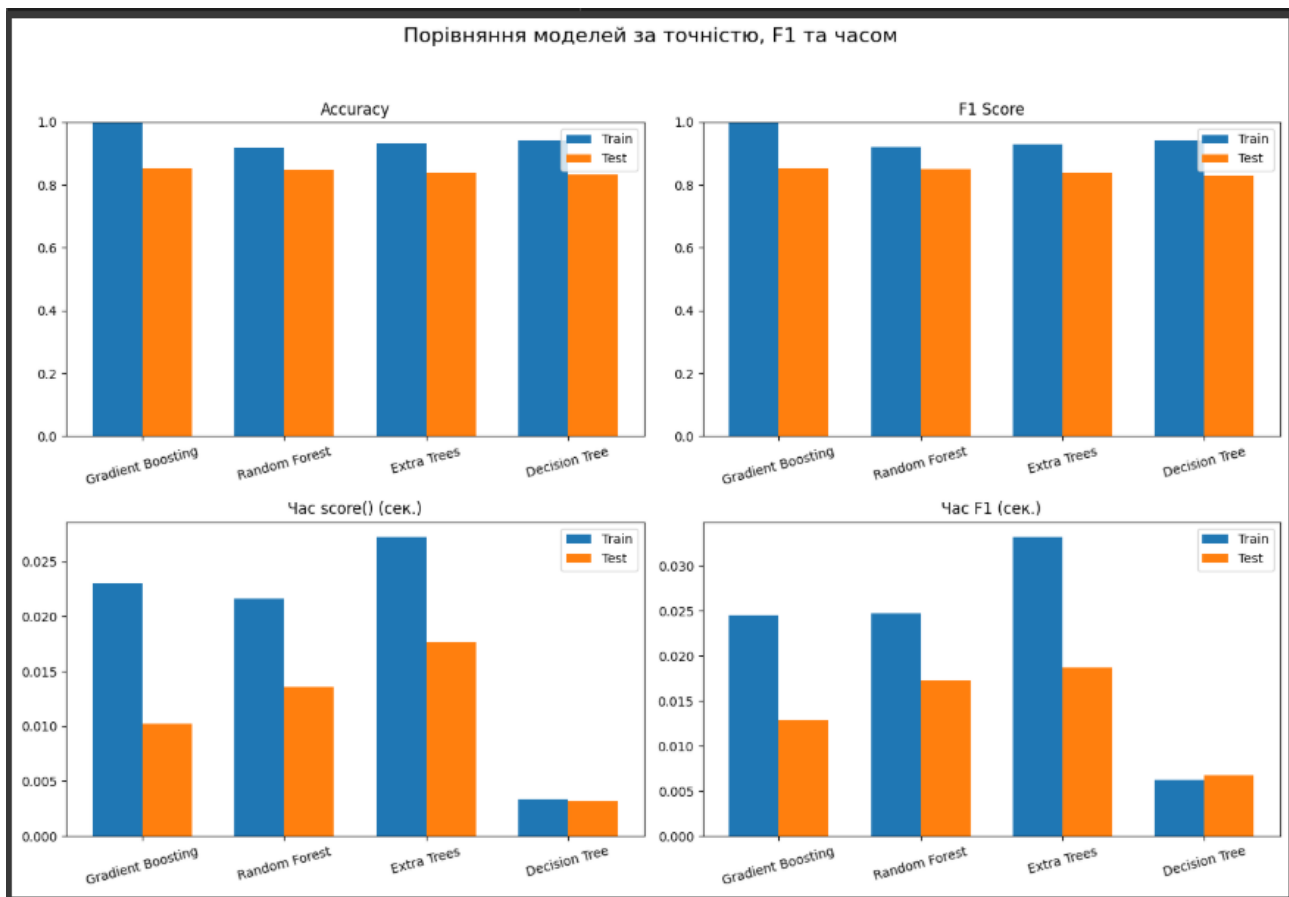


Рисунок 4.15

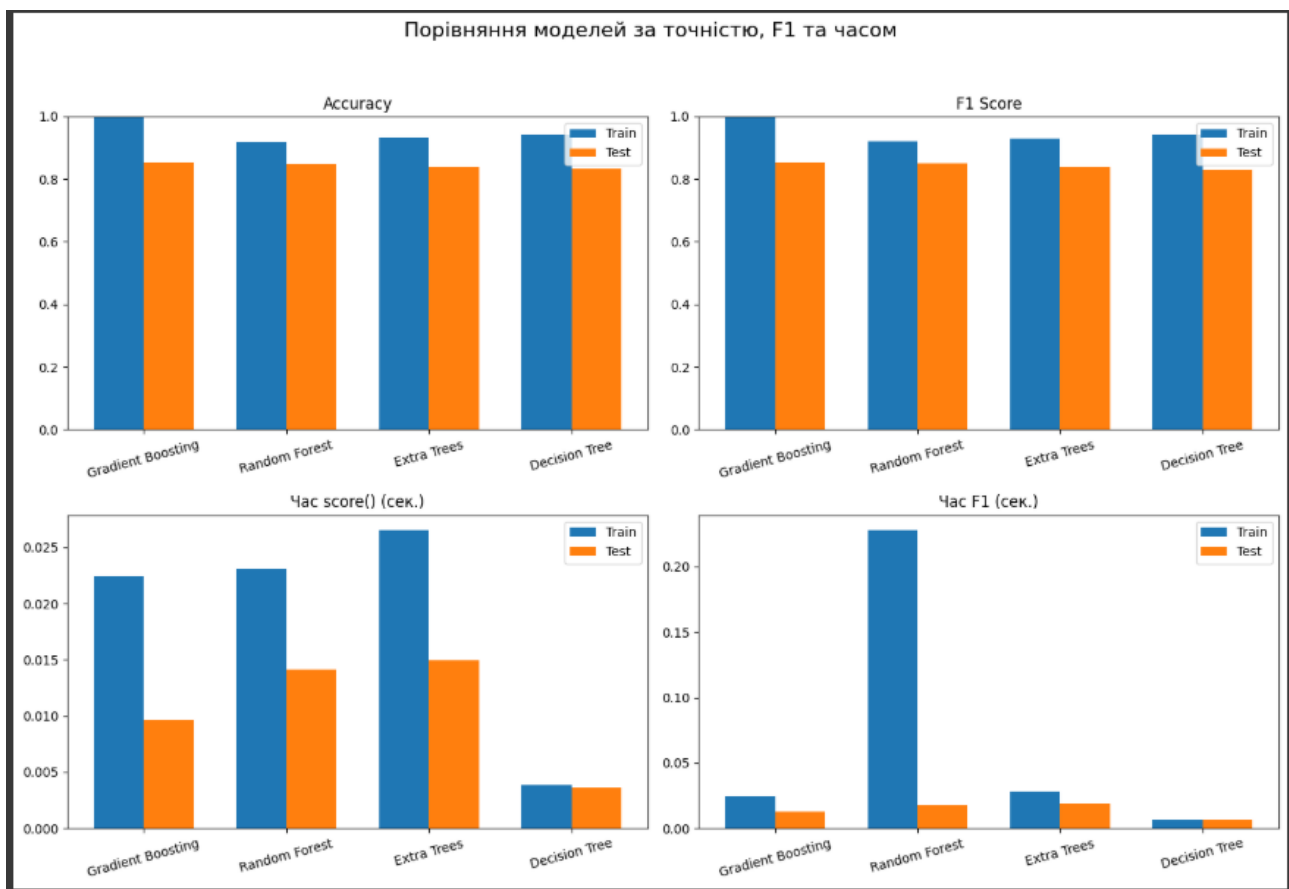


Рисунок 4.16

З отриманих результатів асигасу та f1-score, а також враховуючи рисунки 4.13 – 4.16, можна зробити висновок, що найоптимальнішим методом для категоризації по рівню захворюваності на туберкульоз є Gradient Boosting, адже хоч він і займає багато часу під час тренування моделі, проте на тестових даних показує найкращі результати по точності та одні з найкращих результатів по часу(після decision trees, який проте показує нижчу точність). Методи же Random Forest та Extra Trees є проміжними щодо точності та часу.

ВИСНОВКИ

В результаті виконання курсової роботи було розроблено сховище даних туберкульоз та інших показників типу „зірка”, що містить таблицю фактів та п’ять таблиць вимірів. Написано код для узгодження та фільтрації попередніх даних. Реалізовано ETL процеси для завантаження даних, написано скрипти для імпорту та корегування даних. Розроблено Stage зону. Для реалізації поставленої задачі було використано MySQL версії 8.2.0, мова програмування Python 3.

На основі детального опису та проведеного аналізу предметної області інтелектуального аналізу даних було виділено категорії захворювання на туберкульоз для аналізу, проведено перевірку даних на кореляцію з рівнем захворювання на туберкульоз.

Використано чотири методи для розв’язання задачі категоризації. Обгрунтовано вибір методів для аналізу. В кінці було порівняно ефективність методів та обрано найоптимальніший.

Найкращим є метод Gradient Boosting, проте якщо швидкість тренування моделі є критичною то можна обрати Random Forest або Extra Trees.

Отже, поставлені задачі були виконані.

ПЕРЕЛІК ПОСИЛАНЬ

1. Wikipedia: COVID-19. [Електронний ресурс] – Режим доступу до ресурсу: https://uk.wikipedia.org/wiki/Коронавірусна_хвороба_2019
2. Who: Tuberculosis. [Електронний ресурс] – Режим доступу до ресурсу: <https://www.who.int/health-topics/tuberculosis>
3. Документація мови програмування Python. [Електронний ресурс] – Режим доступу до ресурсу: <https://docs.python.org/3/>
4. Бібліотека Pandas. [Електронний ресурс] – Режим доступу до ресурсу: <https://pandas.pydata.org/>
5. Бібліотека Numpy. [Електронний ресурс] – Режим доступу до ресурсу: <https://numpy.org/>
6. Бібліотека Matplotlib. [Електронний ресурс] – Режим доступу до ресурсу: <https://matplotlib.org/>
7. Бібліотека Seaborn. [Електронний ресурс] – Режим доступу до ресурсу: <https://seaborn.pydata.org/>
8. Бібліотека Scipy. [Електронний ресурс] – Режим доступу до ресурсу: <https://scipy.org/>
9. Бібліотека Time. [Електронний ресурс] – Режим доступу до ресурсу: <https://docs.python.org/uk/3.13/library/time.html>
10. Платформа Google Colaboratory. [Електронний ресурс] – Режим доступу до ресурсу: <https://colab.google/>.
11. Бібліотека Sklearn. [Електронний ресурс] – Режим доступу до ресурсу: <https://scikit-learn.org/stable/>
12. Gradient Boosting. [Електронний ресурс] – Режим доступу до ресурсу: <https://www.ibm.com/think/topics/gradient-boosting>
13. Random Forest Algorithm in Machine Learning. [Електронний ресурс] – Режим доступу до ресурсу: <https://www.geeksforgeeks.org/random-forest-algorithm-in-machine-learning/>

14.What When How ExtraTrees Classifier. [Электронный ресурс] – Режим доступа до ресурсу: <https://towardsdatascience.com/what-when-how-extratrees-classifier-c939f905851c/>

15.Decision Tree. [Электронный ресурс] – Режим доступа до ресурсу: <https://www.geeksforgeeks.org/decision-tree/>

ДОДАТОК А ТЕКСТИ ПРОГРАМНОГО КОДУ

Тексти програмного коду Аналіз зв'язку різних факторів та
рівня захворювання туберкульозом для розв'язання задачі

категоризації
(Найменування програми (документа))

SSD
(Вид носія даних)

19 арк, 33 Кб
(Обсяг програми (документа), арк..)

студента групи ІП-33 ІІ курсу

Піковського Володимира

Мирославовича

PreparingFiles.py

```
import pandas as pd
import numpy as np

# Зчитуємо початкові датасети
input_file = r"D:\Vova\DataAD\RawData\TB_burden_countries_2025-05-18.csv"
output_file = r"D:\Vova\DataAD\CleanData\tuberculosis_stats.csv"

input_file2 = r"D:\Vova\DataAD\RawData\data.csv"
output_file2 = r"D:\Vova\DataAD\CleanData\smoking_stats.csv"

input_file3 =
r"D:\Vova\DataAD\RawData\Global_Development_Indicators_2000_2020.csv"
output_file3 = r"D:\Vova\DataAD\CleanData\economic_stats.csv"

input_file4 = r"D:\Vova\DataAD\RawData\countries.csv"
output_file4 = r"D:\Vova\DataAD\CleanData\social_stats.csv"

df = pd.read_csv(input_file)
df2 = pd.read_csv(input_file2)
df3 = pd.read_csv(input_file3)
df4 = pd.read_csv(input_file4)

# Зводимо назви колонок до одного стандарту
dfs = [df, df2, df3, df4]
for i in range(len(dfs)):
    dfs[i].columns = [col[0].lower() + col[1:] if col else col for col in
dfs[i].columns]
```

```
df, df2, df3, df4 = dfs
```

```
# Видаляємо непотрібні колонки та для зручності перейменовуємо ті що залишилися
```

```
df = df[["country", "year", "g_whoregion", "e_inc_num", "e_pop_num",  
"e_inc_tbhiv_num", "c_cdr", "cfr"]].copy()  
df2 = df2[["country", "age", "gender", "year", "smoking_prevalence"]].copy()  
df3 = df3[["country_name", "year", "gdp_per_capita",  
"health_development_ratio", "human_development_index"]].copy()  
df4 = df4[["country", "language", "religion"]].copy()
```

```
df.rename(columns={  
    "g_whoregion": "world_region",  
    "e_inc_num": "infected_num",  
    "e_pop_num": "population",  
    "e_inc_tbhiv_num": "infected_with_hiv_num",  
    "c_cdr": "case_detection_rate",  
    "cfr": "case_fatality_rate"  
}, inplace=True)
```

```
df3.rename(columns={"country_name": "country"}, inplace=True)
```

```
# Обрізаємо роки що не є між 2000 та 2012 для узгодженості даних
```

```
dfList = [df, df2, df3]
```

```
for i in range(len(dfList)):
```

```
    dfList[i] = dfList[i].copy()
```

```
    dfList[i]['year'] = pd.to_numeric(dfList[i]['year'], errors='coerce')
```

```
    dfList[i] = dfList[i][(dfList[i]['year'] >= 2000) & (dfList[i]['year'] <= 2012)]
```

```
df, df2, df3 = dfList
```

```
# Додаємо нові колонки - відсоток хворих на туберкульоз та відсоток  
хворих на ВІЛ серед хворих на туберкульоз
```

```
df['infected_ratio'] = df['infected_num'] / df['population']
```

```
df['hiv_in_tb_ratio'] = df['infected_with_hiv_num'] / df['infected_num']
```

```
df['hiv_in_tb_ratio'] = pd.to_numeric(df['hiv_in_tb_ratio'], errors='coerce')
```

```
df['hiv_in_tb_ratio'] = df['hiv_in_tb_ratio'].replace([np.inf, -np.inf], np.nan)
```

```
df['hiv_in_tb_ratio'] =
```

```
df.groupby('country')['hiv_in_tb_ratio'].transform(lambda x: x.fillna(x.mean()))
```

```
df['hiv_in_tb_ratio'] = df['hiv_in_tb_ratio'].fillna(df['hiv_in_tb_ratio'].mean())
```

```
# Зберігаємо
```

```
df.to_csv(output_file, index=False)
```

```
df2.to_csv(output_file2, index=False)
```

```
df3.to_csv(output_file3, index=False)
```

```
df4.to_csv(output_file4, index=False)
```

```
# Узгоджуємо країни між датасетами
```

```
clean_files = [output_file, output_file2, output_file3, output_file4]
```

```
country_sets = [set(pd.read_csv(f)['country'].dropna().str.strip()) for f in  
clean_files]
```

```
common_countries = set.intersection(*country_sets)
```

```
all_countries = set.union(*country_sets)
```

```
different_countries = all_countries - common_countries
```

```
common_countries = sorted(common_countries)
```

```
different_countries = sorted(different_countries)
```

```
print(f"Common countries in all files ({len(common_countries)}):")
print(common_countries)
```

```
print(f"\nDifferent countries (not in all files) ({len(different_countries)}):")
print(different_countries)
```

```
for file_path in clean_files:
```

```
    df_temp = pd.read_csv(file_path)
    df_temp['country'] = df_temp['country'].str.strip()
    df_temp = df_temp[df_temp['country'].isin(common_countries)]
    df_temp.to_csv(file_path, index=False)
```

```
# Видаляємо непотрібні колонки та рядки з датасету щодо куріння
```

```
file_path = output_file2
df = pd.read_csv(file_path)
df = df[df['age'] == 'All-ages']
df = df[df['gender'] == 'Both']
df = df.drop(columns=['age', 'gender'])
df.to_csv(file_path, index=False)
```

Analysis.py

```
# -*- coding: utf-8 -*-
"""Untitled0.ipynb
```

Automatically generated by Colab.

Original file is located at

<https://colab.research.google.com/drive/1Cl-rKfYpV43L9fiA0px0qzFL0F0YwqDJ>

"""

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
import time

from google.colab import drive

from sklearn.model_selection import train_test_split, cross_val_score, KFold,
GridSearchCV
from sklearn.linear_model import LinearRegression
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score,
classification_report, silhouette_score, confusion_matrix
from sklearn.ensemble import (
    RandomForestRegressor, GradientBoostingRegressor, ExtraTreesRegressor,
    RandomForestClassifier, ExtraTreesClassifier, GradientBoostingClassifier
)
from sklearn.tree import DecisionTreeRegressor, DecisionTreeClassifier
from sklearn import tree
from sklearn.metrics import f1_score

drive.mount('/content/drive')
filename = "/content/drive/My Drive/fact_health_stats.csv"
df = pd.read_csv(filename, header=0)
```



```
df.info()
```

```
df.describe()
```

```
df.head()
```

```
"""Тепер визначимо, які параметри корелюють з рівнем захворюваності  
на туберкульоз"""
```

```
# Список колонок, які перевірятимемо на кореляцію з infected_ratio
```

```
target = 'infected_ratio'
```

```
features = [
```

```
    'population',
```

```
    'case_detection_rate',
```

```
    'case_fatality_rate',
```

```
    'smoking_prevalence',
```

```
    'gdp_per_capita',
```

```
    'health_development_ratio',
```

```
    'human_development_index',
```

```
    'hiv_in_tb_ratio',
```

```
    'language_id',
```

```
    'religion_id',
```

```
    'region_id'
```

```
]
```

```
correlation_results = []
```

```
# Обчислення кореляцій
```

```
for feature in features:
```

```
    pearson_coef, p_value = stats.pearsonr(df[target], df[feature])
```

```
    correlation_results.append({
```

```
        'feature': feature,
```

```
        'pearson_coef': pearson_coef,
        'p_value': p_value
    })

    print(f"The Pearson Correlation Coefficient between '{target}' and
    '{feature}' is {pearson_coef:.4f} with a P-value of {p_value:.4g}")

    corr_df = pd.DataFrame(correlation_results).sort_values(by='pearson_coef',
    key=abs, ascending=False)
```

```
# Побудова графіка
plt.figure(figsize=(10, 6))
sns.barplot(
    data=corr_df,
    x='pearson_coef',
    y='feature',
    hue='feature',
    palette='coolwarm',
    dodge=False,
    legend=False
)
plt.axvline(0, color='gray', linestyle='--')
plt.title('Кореляція змінних з infected_ratio')
plt.xlabel('Коефіцієнт кореляції Пірсона')
plt.ylabel('Змінна')
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
# Гістограма для infected_ratio
fig, axes = plt.subplots(1, 1, figsize=(8, 5))
```

```
axes.hist(df["infected_ratio"].dropna(), bins=50, color="green",
edgecolor="black", alpha=0.7)

axes.set_title("Гістограма (Інфікованість туберкульозом)", fontsize=12)
axes.set_xlabel("Рівень інфікованості (infected_ratio)")
axes.set_ylabel("Кількість країн")

plt.tight_layout()
plt.show()

# Функція для класифікації рівня інфікованості
def classify_infected_level(ratio):
    if ratio < 0.001:
        return 1
    elif 0.001 <= ratio < 0.003:
        return 2
    elif 0.003 <= ratio < 0.005:
        return 3
    else: # 0.005 і більше
        return 4

# Додаємо нову колонку до DataFrame
df['infected_level'] = df['infected_ratio'].apply(classify_infected_level)
df.head(250)

# Готуємо дані до задачі класифікації, будемо намагатися визначити
infected_level

# Беремо ті параметри, що найбільше корелюють з infection_rate
all_features=pd.get_dummies(df[['case_detection_rate',
```

```

        'case_fatality_rate', 'health_development_ratio',
        'human_development_index', 'hiv_in_tb_ratio']])

all_features[['infected_level']] = df[['infected_level']]

# Ділимо дані на навчальну та тестову вибірки
df_train, df_test = train_test_split(
    all_features,
    test_size=0.2,
    random_state=1
)

x_train = df_train[['case_detection_rate',
    'case_fatality_rate', 'health_development_ratio',
    'human_development_index', 'hiv_in_tb_ratio']]
y_train = df_train[['infected_level']]

x_test = df_test[['case_detection_rate',
    'case_fatality_rate', 'health_development_ratio',
    'human_development_index', 'hiv_in_tb_ratio']]
y_test = df_test[['infected_level']]

# Підбираємо параметри для GradientBooster
gb = GradientBoostingClassifier(random_state=1)

param_grid = {
    'n_estimators': [100],
    'learning_rate': [0.05, 0.1],
    'max_depth': [3, 5]
}

grid_search = GridSearchCV(estimator=gb, param_grid=param_grid,

```

```

        cv=5, scoring='accuracy', n_jobs=-1, verbose=2)
grid_search.fit(x_train, y_train)

# Виведення результатів
print("Best parameters found:", grid_search.best_params_)
print("Best cross-validation accuracy:", grid_search.best_score_)

# Підбираємо параметр random_state та будуємо графік точності
accur = []
max_random = 20
for k in range(1, max_random + 1):
    gradboost = GradientBoostingClassifier(learning_rate=0.1, max_depth=5,
n_estimators=100, random_state=k)
    gradboost.fit(x_train, y_train)
    accur.append(gradboost.score(x_test, y_test))

plt.figure(figsize=(10, 8))
plt.plot(range(1, max_random + 1), accur)
plt.xticks(range(0, max_random + 1, 1))
plt.xlabel('max_random')
plt.ylabel('mean accuracy')
plt.grid(linestyle='--')
plt.show()

# Підставляємо визначені параметри та оцінюємо точність моделі
gradboost = GradientBoostingClassifier(learning_rate=0.1, max_depth=5,
n_estimators=100)
gradboost.fit(x_train, y_train)
print('mean accuracy = ', gradboost.score(x_test, y_test))

```

```
# Підбираємо параметр max_depth та будуємо графік точності
accur = []
max_kernels = 20
for k in range(1, max_kernels + 1):
    randomforest = RandomForestClassifier(max_depth=k, random_state=1)
    randomforest.fit(x_train, y_train)
    accur.append(randomforest.score(x_test, y_test))
```

```
plt.figure(figsize=(10, 8))
plt.plot(range(1, max_kernels + 1), accur)
plt.xticks(range(0, max_kernels + 1, 1))
plt.xlabel('max_depth')
plt.ylabel('mean accuracy')
plt.grid(linestyle='--')
plt.show()
```

```
# Підбираємо параметр random_state та будуємо графік точності
accur = []
max_random = 20
for k in range(1, max_random + 1):
    randomforest = RandomForestClassifier(max_depth=7, random_state=k)
    randomforest.fit(x_train, y_train)
    accur.append(randomforest.score(x_test, y_test))
```

```
plt.figure(figsize=(10, 8))
plt.plot(range(1, max_random + 1), accur)
plt.xticks(range(0, max_random + 1, 1))
plt.xlabel('max_random')
plt.ylabel('mean accuracy')
plt.grid(linestyle='--')
```

```
plt.show()
```

```
# Підставляємо визначені параметри та оцінюємо точність моделі
randomforest = RandomForestClassifier(max_depth=7, random_state=1)
random_scores = cross_val_score(randomforest, x_train, y_train, cv=5)
random_scores.mean()
randomforest.fit(x_train, y_train)
print('mean accuracy = ', randomforest.score(x_test, y_test))
```

```
# Підбираємо параметр max_depth та будуємо графік точності
accur = []
max_kernels = 20
for k in range(1, max_kernels + 1):
    extratrees = ExtraTreesClassifier(max_depth=k, random_state=1)
    extratrees.fit(x_train, y_train)
    accur.append(extratrees.score(x_test, y_test))
```

```
plt.figure(figsize=(10, 8))
plt.plot(range(1, max_kernels + 1), accur)
plt.xticks(range(0, max_kernels + 1, 1))
plt.xlabel('max_depth')
plt.ylabel('mean accuracy')
plt.grid(linestyle='--')
plt.show()
```

```
# Підбираємо параметр random_state та будуємо графік точності
accur = []
max_random = 20
for k in range(1, max_random + 1):
    extratrees = ExtraTreesClassifier(max_depth=10, random_state=k)
```

```
extratrees.fit(x_train, y_train)
accur.append(extratrees.score(x_test, y_test))
```

```
plt.figure(figsize=(10, 8))
plt.plot(range(1, max_random + 1), accur)
plt.xticks(range(0, max_random + 1, 1))
plt.xlabel('max_random')
plt.ylabel('mean accuracy')
plt.grid(linestyle='--')
plt.show()
```

```
# Підставляємо визначені параметри та оцінюємо точність моделі
extratrees = ExtraTreesClassifier(max_depth=10, random_state = 10)
extra_scores = cross_val_score(extratrees, x_train, y_train, cv=5)
extra_scores.mean()
extratrees.fit(x_train, y_train)
print('mean accuracy = ', extratrees.score(x_test, y_test))
```

```
# Підбираємо параметр max_depth та будуємо графік точності
accur = []
max_kernels = 20
for k in range(1, max_kernels + 1):
    decision_tree = DecisionTreeClassifier(max_depth=k, random_state=1)
    decision_tree.fit(x_train, y_train)
    accur.append(decision_tree.score(x_test, y_test))
```

```
plt.figure(figsize=(10, 8))
plt.plot(range(1, max_kernels + 1), accur)
plt.xticks(range(0, max_kernels + 1, 1))
plt.xlabel('max_depth')
```



```
plt.ylabel('mean accuracy')
plt.grid(linestyle='--')
plt.show()
```

```
# Підбираємо параметр random_state та будуємо графік точності
accur = []
max_random = 20
for k in range(1, max_random + 1):
    decision_tree = DecisionTreeClassifier(max_depth=9, random_state=k)
    decision_tree.fit(x_train, y_train)
    accur.append(decision_tree.score(x_test, y_test))
```

```
plt.figure(figsize=(10, 8))
plt.plot(range(1, max_random + 1), accur)
plt.xticks(range(0, max_random + 1, 1))
plt.xlabel('max_random')
plt.ylabel('mean accuracy')
plt.grid(linestyle='--')
plt.show()
```

```
# Підставляємо визначені параметри та оцінюємо точність моделі
decision_tree = DecisionTreeClassifier(max_depth=9, random_state=8)
tree_scores = cross_val_score(decision_tree, x_train, y_train, cv=5)
tree_scores.mean()
decision_tree.fit(x_train, y_train)
print('mean accuracy = ', decision_tree.score(x_test, y_test))
```

```
#Порівняння точностей моделей не лише на тестових вибірках, а й на
навчальних
models = {
```

```

'Gradient Boosting': gradboost,
'Random Forest': randomforest,
'Extra Trees': extratrees,
'Decision Tree': decision_tree
}

print("Accuracy on training data")
for name, model in models.items():
    start = time.time()
    acc_train = model.score(x_train, y_train)
    duration = time.time() - start
    print(f"{name} : {acc_train:.4f} (Time: {duration:.4f} sec)")

print("\nAccuracy on test data")
for name, model in models.items():
    start = time.time()
    acc_test = model.score(x_test, y_test)
    duration = time.time() - start
    print(f"{name} : {acc_test:.4f} (Time: {duration:.4f} sec)")

print("\nF1 Score on training data")
for name, model in models.items():
    start = time.time()
    y_pred_train = model.predict(x_train)
    f1_train = f1_score(y_train, y_pred_train, average='weighted')
    duration = time.time() - start
    print(f"{name} : {f1_train:.4f} (Time: {duration:.4f} sec)")

print("\nF1 Score on test data")
for name, model in models.items():

```

```
start = time.time()
y_pred_test = model.predict(x_test)
f1_test = f1_score(y_test, y_pred_test, average='weighted')
duration = time.time() - start
print(f'{name} : {f1_test:.4f} (Time: {duration:.4f} sec)')
```

#Побудова графіків порівняння моделей

```
models = {
    'Gradient Boosting': gradboost,
    'Random Forest': randomforest,
    'Extra Trees': extratrees,
    'Decision Tree': decision_tree
}
```

```
metrics = {
    'model': [],
    'accuracy_train': [],
    'accuracy_test': [],
    'f1_train': [],
    'f1_test': [],
    'time_accuracy_train': [],
    'time_accuracy_test': [],
    'time_f1_train': [],
    'time_f1_test': []
}
```

Розрахунок метрик

```
for name, model in models.items():
    metrics['model'].append(name)
```

```
start = time.time()
acc_train = model.score(x_train, y_train)
metrics['time_accuracy_train'].append(time.time() - start)
metrics['accuracy_train'].append(acc_train)
```

```
start = time.time()
acc_test = model.score(x_test, y_test)
metrics['time_accuracy_test'].append(time.time() - start)
metrics['accuracy_test'].append(acc_test)
```

```
start = time.time()
y_pred_train = model.predict(x_train)
f1_train = f1_score(y_train, y_pred_train, average='weighted')
metrics['time_f1_train'].append(time.time() - start)
metrics['f1_train'].append(f1_train)
```

```
start = time.time()
y_pred_test = model.predict(x_test)
f1_test = f1_score(y_test, y_pred_test, average='weighted')
metrics['time_f1_test'].append(time.time() - start)
metrics['f1_test'].append(f1_test)
```

```
labels = metrics['model']
x = np.arange(len(labels))
width = 0.35
```

```
fig, axs = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle("Порівняння моделей за точністю, F1 та часом", fontsize=16)
```

```
axs[0, 0].bar(x - width/2, metrics['accuracy_train'], width, label='Train')
```

```
axs[0, 0].bar(x + width/2, metrics['accuracy_test'], width, label='Test')
axs[0, 0].set_title('Accuracy')
axs[0, 0].set_xticks(x)
axs[0, 0].set_xticklabels(labels, rotation=15)
axs[0, 0].set_ylim(0, 1)
axs[0, 0].legend()
```

```
axs[0, 1].bar(x - width/2, metrics['f1_train'], width, label='Train')
axs[0, 1].bar(x + width/2, metrics['f1_test'], width, label='Test')
axs[0, 1].set_title('F1 Score')
axs[0, 1].set_xticks(x)
axs[0, 1].set_xticklabels(labels, rotation=15)
axs[0, 1].set_ylim(0, 1)
axs[0, 1].legend()
```

```
axs[1, 0].bar(x - width/2, metrics['time_accuracy_train'], width, label='Train')
axs[1, 0].bar(x + width/2, metrics['time_accuracy_test'], width, label='Test')
axs[1, 0].set_title('Ώac score() (cek.)')
axs[1, 0].set_xticks(x)
axs[1, 0].set_xticklabels(labels, rotation=15)
axs[1, 0].legend()
```

```
axs[1, 1].bar(x - width/2, metrics['time_f1_train'], width, label='Train')
axs[1, 1].bar(x + width/2, metrics['time_f1_test'], width, label='Test')
axs[1, 1].set_title('Ώac F1 (cek.)')
axs[1, 1].set_xticks(x)
axs[1, 1].set_xticklabels(labels, rotation=15)
axs[1, 1].legend()
```

```
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
```

```
plt.show()
```